Arquitectura y Documentación de Aplicaciones Backend

Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura por capas

Express Router

Estructura de carpetas

Ejemplo aplicado: frutas y MongoDB

La importancia de documentar

Formas de documentar en Node.js

Swagger UI

Postman

JS Docs

Markdown

Sintaxis de MarkDown

Listas de tareas y tablas

Agregar código

Agregar código con lenguaje

Referenciar hipervínculos

agregar diagramas

Estructura de la documentación API RESTFUL

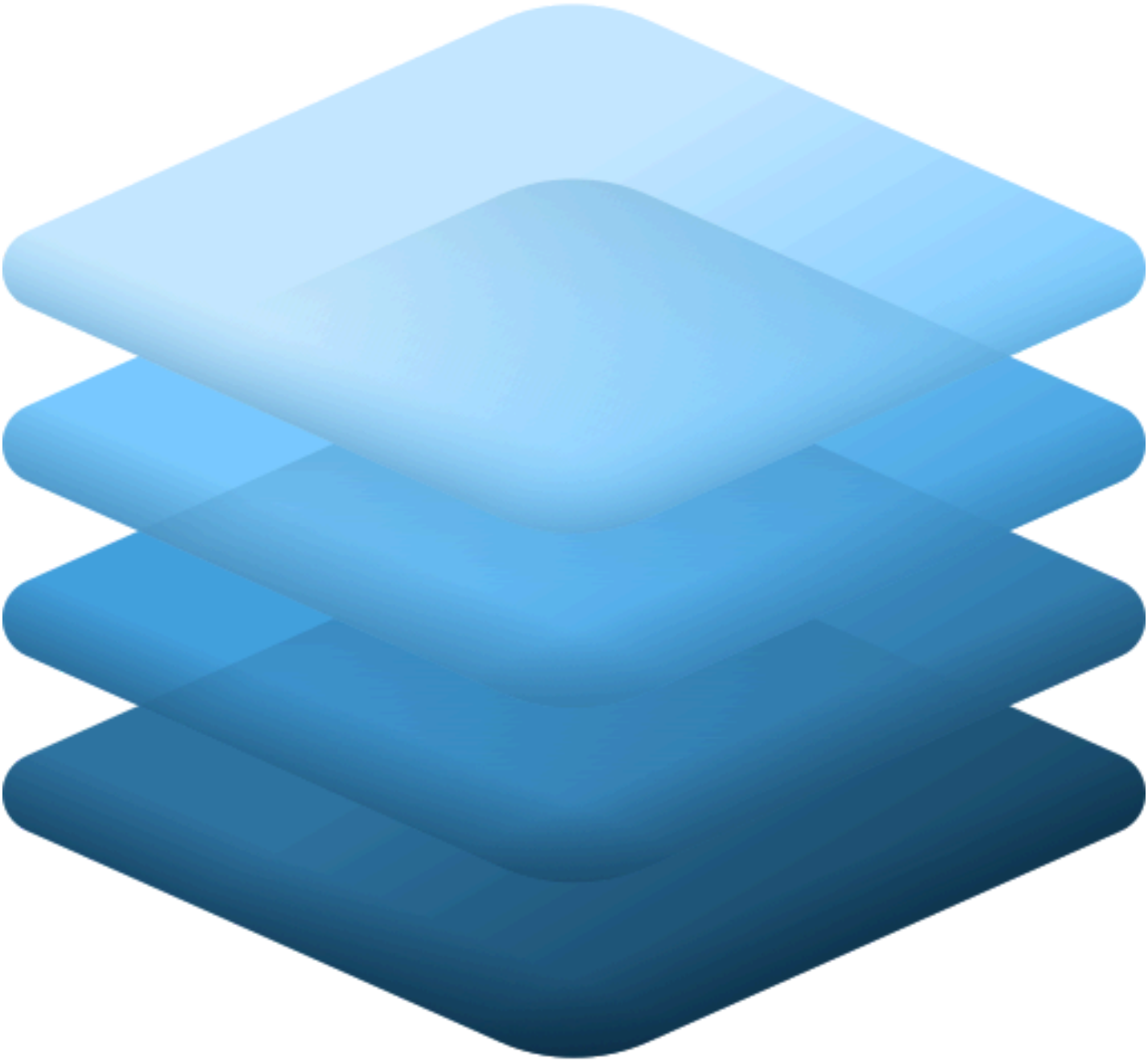
Grabación de la Clase

Página actualizada 25 jun 2026 Denunciar abuso

06-19.mp4

Arquitectura por capas

ha demostrado su valía una y otra vez.



En el mundo del desarrollo de software, la organización y la estructura son elementos cruciales para el éxito de cualquier proyecto, por ello, la **Arquitectura por Capas** se destaca como una metodología sólida y probada que se ha convertido en un pilar en la industria.

Página actualizada 25 jun 2026 Denunciar abuso

proyectos de todos los tamaños y complejidades. Por lo cual, dentro de sus beneficios, podemos destacar:

- versatilidad
- capacidad de adaptación
- elección confiable y escalable

Su versatilidad y capacidad para adaptarse a una amplia gama de aplicaciones, tanto backend como también frontend, la convierten en una elección confiable para los desarrolladores y arquitectos de software.

Entender este concepto es fundamental porque **nos ayudará a escalar fácilmente los proyectos**, **integrando partes nuevas** en estos **sin tener que hacer una reingeniería completa** sobre las características ya creadas y funcionales.

numerosas y poderosas. Al descomponer nuestras aplicaciones en capas interconectadas, logramos una serie de beneficios clave:

Modularización: La descomposición en capas permite una organización más limpia y modular de la lógica de la aplicación.

Escalabilidad: Facilita la escalabilidad al permitir la adición o modificación de capas de manera independiente.

Mantenibilidad: Mejora la mantenibilidad al aislar y gestionar las partes de la aplicación de manera más eficiente.

Reusabilidad: Las capas pueden ser reutilizadas en diferentes partes de la aplicación o incluso en proyectos diferentes.

Colaboración efectiva: Facilita la colaboración en equipos de desarrollo al definir claramente las responsabilidades de cada capa.

Abstracción: Permite abstraer detalles complejos y centrarse en la funcionalidad de cada capa.

Reducción de la Complejidad: Reduce la complejidad global de la aplicación al dividirla en partes manejables.

Claridad en la Arquitectura: Proporciona una estructura clara que mejora la comprensión de la aplicación por parte de los desarrolladores.

Mayor Adaptabilidad: Facilita la adaptación a cambios y actualizaciones futuras en la aplicación.

Como podemos apreciar, sobran beneficios en este terreno. Prácticamente todos ellos

Página actualizada 25 jun 2026 Denunciar abuso

son aplicables al ecosistema backend, como también al ecosistema de desarrollo



Arquitectura y Documentación …

Para comprender la Arquitectura por Capas en su totalidad, **debemos examinar sus componentes fundamentales**. Cada capa cumple un papel específico en el flujo de datos y la lógica de la aplicación:

Capa de Presentación: Esta capa se encarga de la interacción con los usuarios y la presentación de la información. Es la interfaz visible de la aplicación y maneja la entrada y salida de datos.

En frontend esta capa corresponde a la interfaz gráfica con la cual interactúa el usuario (UI).

En backend esta capa corresponde a la interfaz con la cual interactúa el desarrollador frontend; en nuestro caso, la API RESTFUL o Endpoints a través de los cuales obtiene información.

Capa de Lógica de Negocio: Alberga la lógica central de la aplicación. Aquí se definen las reglas y operaciones que gobiernan el funcionamiento de la aplicación, entre ellos:

- Procesamiento de datos
- Reglas de negocio
- Gestión de usuario
- Integración de servicios externos
- Flujo de trabajo

Página actualizada 25 jun 2026 Denunciar abuso

recibidos desde la capa de presentación y la capa de acceso a datos. Esto puede incluir validaciones, cálculos y transformaciones de datos.

Reglas de negocio: Aquí se definen y aplican las reglas de negocio que rigen el comportamiento de la aplicación. Esto puede incluir políticas de acceso, cálculos de precios, lógica de descuentos y otros aspectos específicos de la actividad de una empresa.

Gestión de Usuarios: La capa de lógica de negocio maneja la autenticación y la gestión de usuarios, incluyendo el registro, inicio de sesión, autorización y gestión de perfiles de usuario.

Integración de Servicios Externos: Si la aplicación se comunica con servicios externos, como APIs de terceros, esta capa se encarga de la integración y la gestión de las solicitudes y respuestas.

Validación de Datos: La capa de lógica de negocio valida los datos antes de realizar cambios en la base de datos o responder a solicitudes de los usuarios. Esto garantiza la integridad de los datos y la seguridad de la aplicación.

Cuando hay una aplicación frontend y una aplicación backend que deben intercambiar datos de forma fluída, se suele decidir cuál de estos dos escenarios toma la responsabilidad de validar los datos antes de almacenarlos o modificarlos en una base de datos… 👇

… en algunos casos, tanto el frontend como el backend pueden validarlos. Sería como un doble check-in.

Pero, en aplicaciones que poseen:

Página actualizada 25 jun 2026 Denunciar abuso

- procesos complejos

se torna conveniente que el frontend se ocupe de las validaciones. Así, el backend, sólo se maneja del guardado o modificación de los mismos.

Procesamiento de datos: aquí se maneja la lógica que se aplica a los datos recibidos desde la capa de presentación y la capa de acceso a datos. Esto puede incluir validaciones, cálculos y transformaciones de datos.

Reglas de negocio: Aquí se definen y aplican las reglas de negocio que rigen el comportamiento de la aplicación. Esto puede incluir políticas de acceso, cálculos de precios, lógica de descuentos y otros aspectos específicos de la actividad de la empresa.

Como podemos ver, el modelo de arquitectura basado en capas es un punto clave a la hora de desarrollar aplicaciones backend.

Ahora que tenemos una experiencia casi completa con todo lo que concierne al manejo y responsabilidad de un backend, y conocimos a casi todos los actores principales que se integran a este (*cookies, sesiones, almacenamiento, bb.dd, encriptación, validación, etc.*) es importante comenzar a involucrarnos en este universo.

Express Router

aplicaciones web de Node.js.



Representa un enfoque lógico y organizado para definir rutas y gestionar la lógica de manejo de solicitudes en nuestras aplicaciones. Exploremos a continuación **qué es Express Router**, la **funcionalidad que aporta** y **cómo debemos considerar su integración** en nuestros desarrollos con Node.js

Página actualizada 25 jun 2026 Denunciar abuso

enfoque modular y estructurado.

En lugar de definir todas las rutas y manejadores de solicitudes en un único archivo, Express Router nos ayuda a dividir estas definiciones en múltiples módulos o archivos independientes.

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('¡Bienvenido a la página de inicio!')
})

app.get('/about', (req, res) => {
  res.send('Esto es la página "Acerca de nosotros".')
})

app.get('/contact', (req, res) => {
  res.send('Póngase en contacto con nosotros en contact@example.com')
})

app.post('/contact', (req, res) => {
  const { nombre, mensaje } = req.body
  res.send(`Gracias, ${nombre}. Hemos recibido tu mensaje: "${mensaje}"`)
})

app.listen(port, () => console.log(`Servidor escuchando el puerto ${port}`))
```

Página actualizada 25 jun 2026 Denunciar abuso

tipo GET y una ruta del tipo POST.

Esta estructura es la que venimos trabajando hasta ahora.

```
const express = require('express')
const app = express()
const router = express.Router()

router.get('/', (req, res) => {
  res.send('¡Bienvenido a la página de inicio!')
})

router.get('/about', (req, res) => {
  res.send('Esto es la página "Acerca de nosotros".')
})

router.get('/contact', (req, res) => {
  res.send('Póngase en contacto con nosotros en contact@example.com')
})

router.post('/contact', (req, res) => {
  const { nombre, mensaje } = req.body
  res.send(`Gracias, ${nombre}. Hemos recibido tu mensaje: "${mensaje}"`)
})

app.use('/app', router)

app.listen(3000, () => {
  console.log('Servidor escuchando el puerto 3000');
})
```

Aquí un ejemplo de la misma estructura, pero aplicando Express Router.

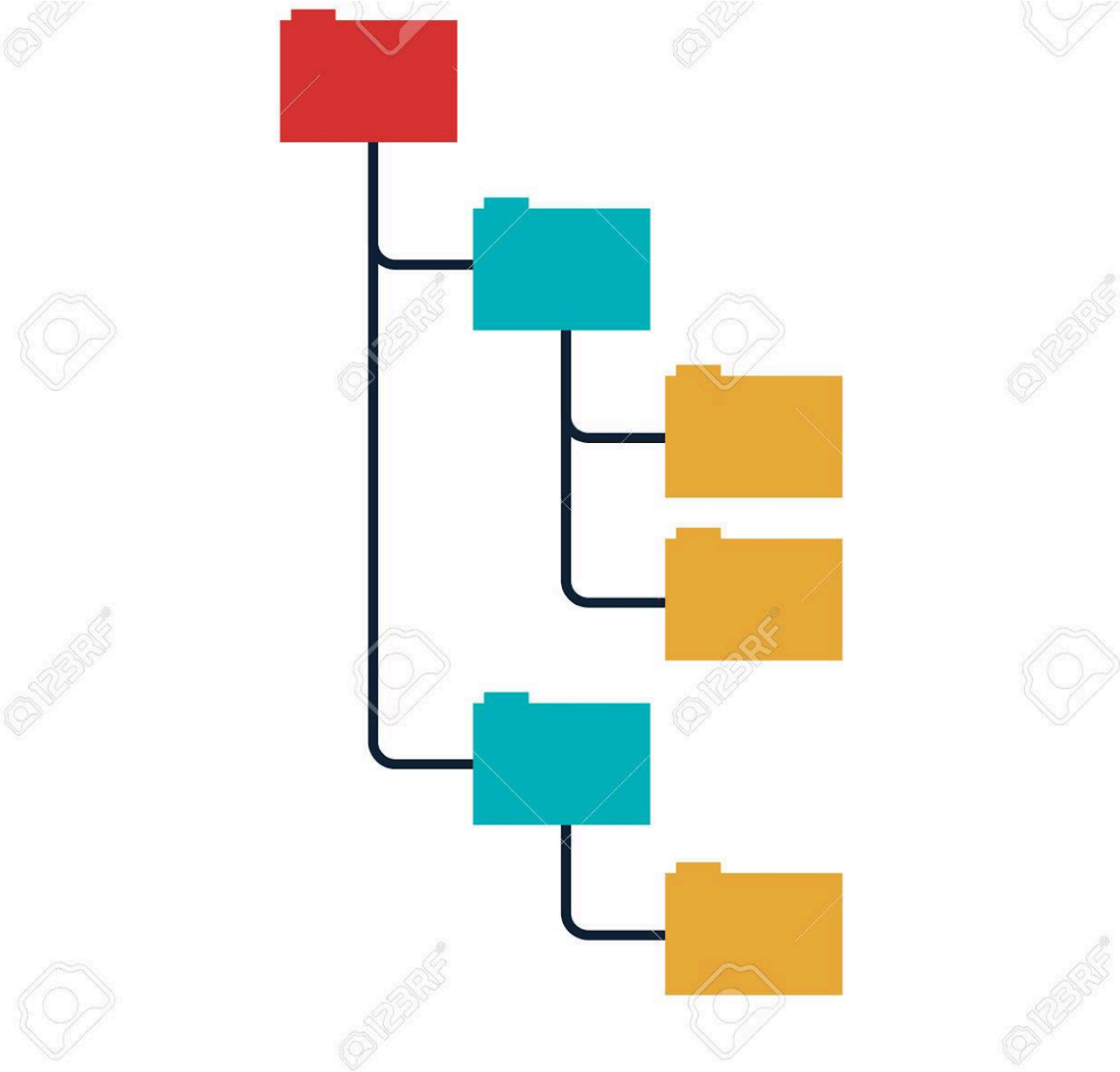
Si bien se asimila bastante a lo trabajado hasta ahora, Express Router nos ayuda bastante a aplicar un modelo de arquitectura por capas en nuestro proyecto backend.

Modularización: Nos permite dividir nuestra aplicación en módulos o controladores independientes, lo que facilita la organización y el mantenimiento de nuestro código.

Separación de Responsabilidades: Al separar rutas y controladores, podemos mantener una clara separación de responsabilidades en nuestra aplicación, lo que facilita la colaboración en equipos de desarrollo.

Manejo de Rutas: Express Router ofrece métodos para definir rutas HTTP (GET, POST, PUT, DELETE, etc.) y asociar controladores a estas rutas, lo que facilita la gestión de las solicitudes entrantes.

Estructura de carpetas

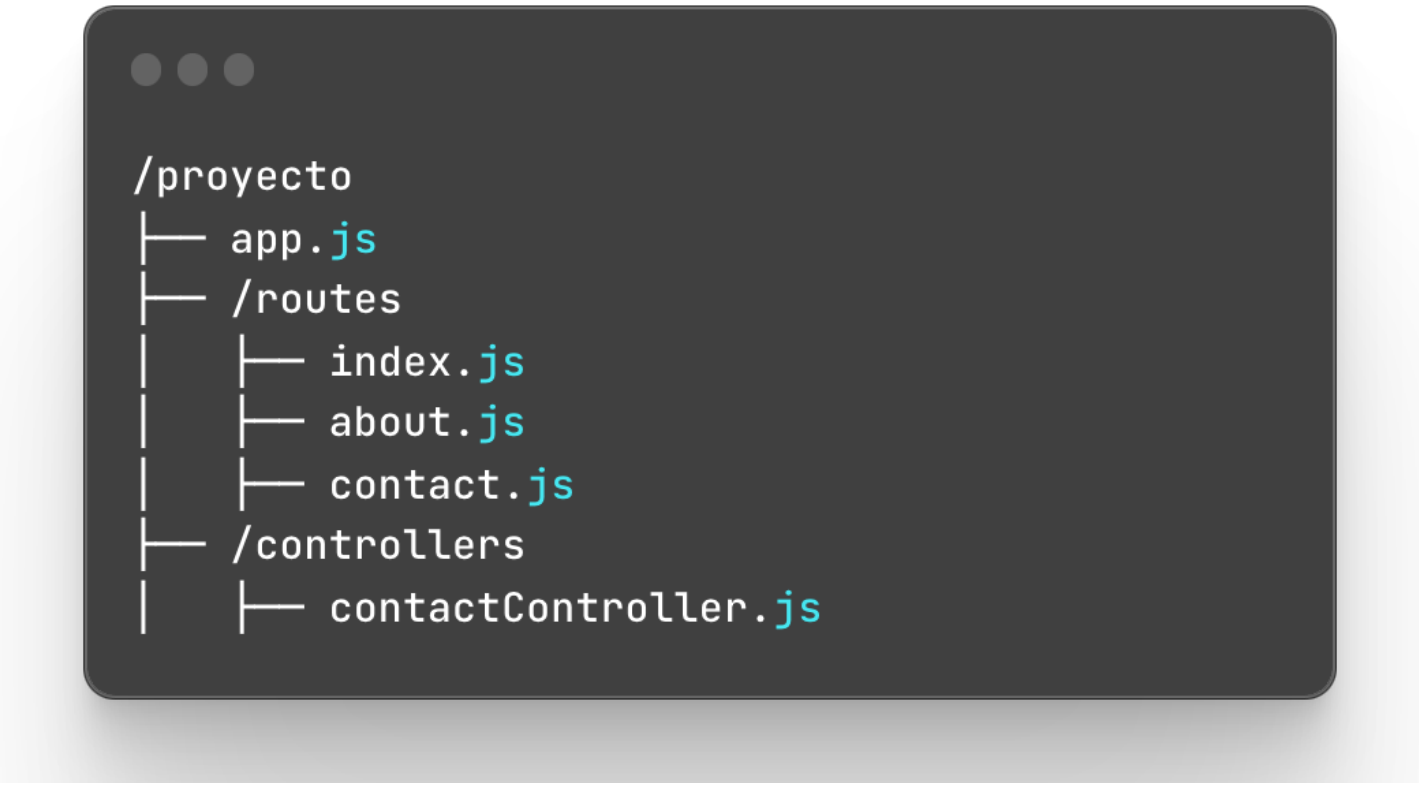


Para organizar el código de este ejemplo utilizando la estructura de carpetas de acuerdo a una arquitectura por capas, podemos pensar en un modelo que separe por completo toda la lógica de cada uno de los endpoints de nuestra aplicación.

Página actualizada 25 jun 2026 Denunciar abuso

manera lógica.

Aquí tenemos un ejemplo de cómo sería un escenario aproximado para el contenido de nuestro proyecto.



Arquitectura y Documentación …

routes/index.js

```
const express = require('express')
const router = express.Router()

router.get('/', (req, res) => {
    res.send('¡Bienvenido a la página de inicio!')
})

module.exports = router
```

routes/about.js

```
const express = require('express')
const router = express.Router()

router.get('/', (req, res) => {
    res.send('Esto es la página "Acerca de nosotros".')
})

module.exports = router
```

routes/contact.js

```
const express = require('express')
const router = express.Router()
```

Página actualizada 25 jun 2026 Denunciar abuso

```
res.send('Póngase en contacto con nosotros en contact@example.com')
```

```
  const { nombre, mensaje } = req.body
  res.send(`Gracias, ${nombre}. Hemos recibido tu mensaje: "${mensaje}"`)
})

module.exports = router
```

La subcarpeta **/controllers/** definimos los posibles controladores, correspondientes para cada ruta.

controllers/contactController.js

```
const contactDatabase = []

function processContactForm(req, res) {
  const { nombre, email, mensaje } = req.body
  if (!nombre || !email || !mensaje) {
      return res.status(400).send('Todos los campos son obligatorios.')
  }

  const nuevoMensaje = {
    nombre,
    email,
    mensaje,
    fecha: new Date(),
  };
  contactDatabase.push(nuevoMensaje)

  res.status(200).send('Gracias por tu mensaje. Te contactaremos pronto.')
}

module.exports = { processContactForm }
```

En este ejemplo, el controlador **contactController.js** contiene una función llamada **processContactForm** que se encarga de procesar el formulario de contacto.

Realiza una validación básica de los campos del formulario, almacena el mensaje en una simulación de base de datos y responde con un mensaje de agradecimiento al remitente.

Página actualizada 25 jun 2026 Denunciar abuso

archivos.

controllers/contactController.js

```
const express = require('express')
const app = express()
const port = 3000

// Importamos las rutas desde sus archivos
const indexRouter = require('./routes/index')
const aboutRouter = require('./routes/about')
const contactRouter = require('./routes/contact')

// Montamos las rutas en la aplicación
app.use('/', indexRouter)
app.use('/about', aboutRouter)
app.use('/contact', contactRouter)

app.listen(port, () => {
    console.log(`El servidor está escuchando en el puerto ${port}`)
})
```

Aquí referenciamos e instanciamos a Express JS, importamos las rutas desde cada uno de sus archivos, y finalmente montamos las rutas en la aplicación principal.

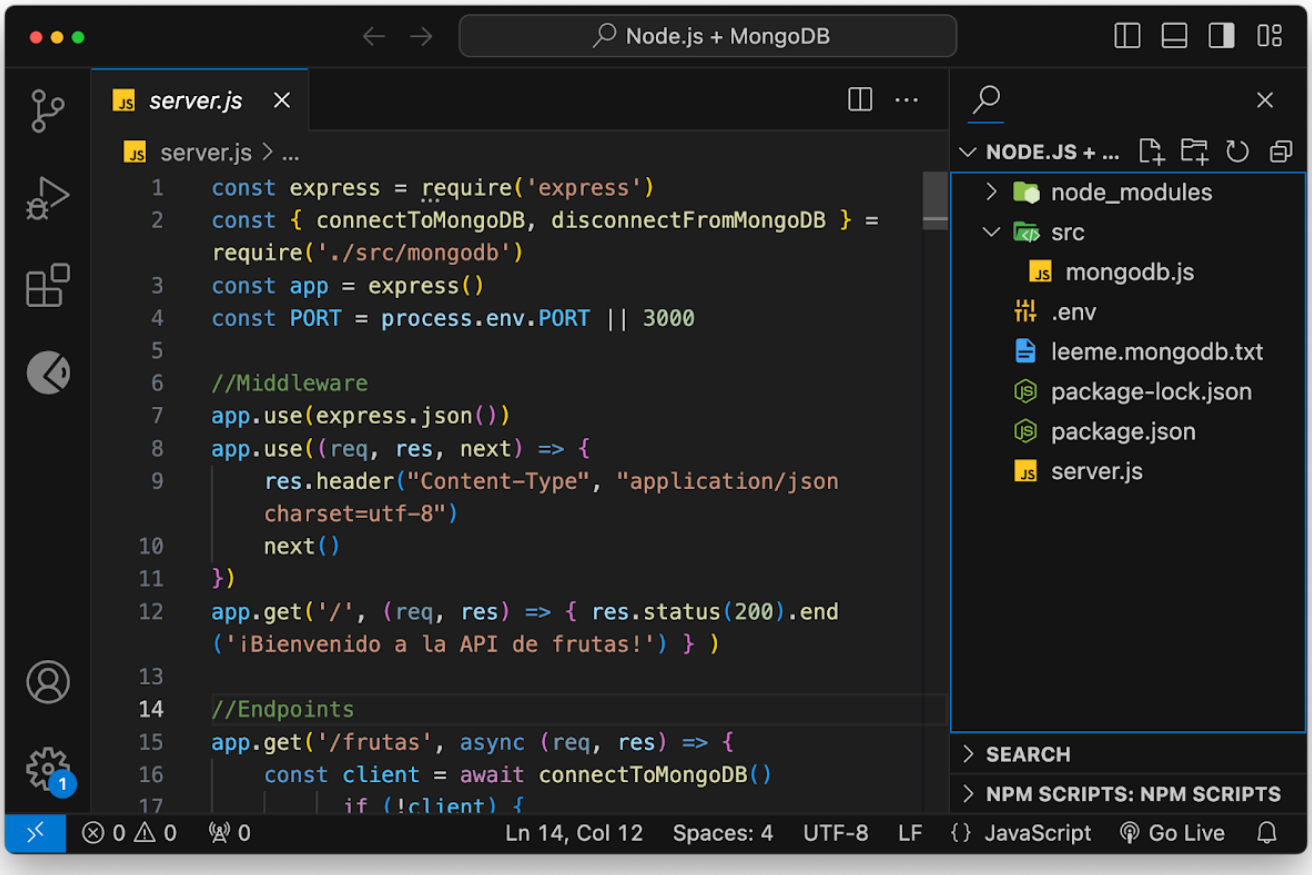
En este último escenario, podemos apreciar el uso de MiddleWares para poder montar cada una de las rutas.

Ejemplo aplicado: frutas y MongoDB

MongoDB, como base de datos.

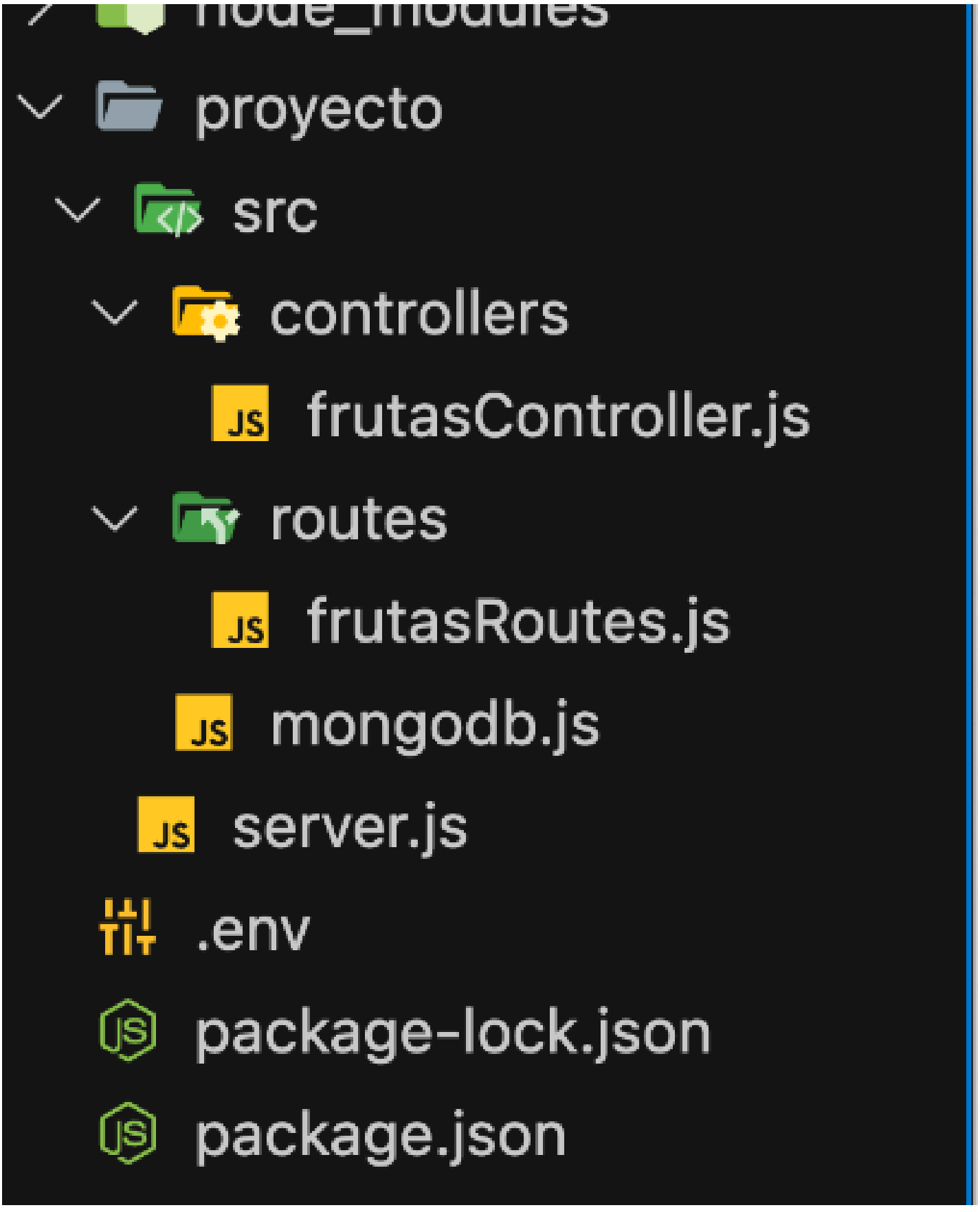
Aquí tenemos representada su estructura original, simple, y basada en, solamente, dos archivos:

- **server.js**
- **/src/mongodb.js**



Router, y la cual aplica el modelo denominado **Arquitectura por Capas**.

```
/proyecto
├── server.js
├── /src
│   ├── /routes
│   │   ├── frutasRoutes.js
│   ├── /controllers
│   │   ├── frutasController.js
│   ├── mongodb.js
```


node_modules
proyecto
src
controllers
frutasController.js
routes
frutasRoutes.js
mongodb.js
server.js
.env
package-lock.json
package.json

/frutas, en un único archivo.

Si bien lo ideal sería separarlo en un archivo por cada endpoint, por cuestiones prácticas no realizaremos este paso pero, **ten presente que es un punto fundamental el aplicar una correcta estructuración del mismo atomizando al máximo posible sus módulos**.

Cada endpoint originalmente creado, se convertirá en el módulo controlador, en una función asincrónica.

Recordemos que esto es un punto clave cuando interactuamos con bases de datos.

src/controllers/frutasController.js

```
const { connectToMongoDB, disconnectFromMongoDB } = require('../mongodb')

async function getFrutas(req, res) {
  const client = await connectToMongoDB()
        if (!client) {
            res.status(500).send('Error al conectarse a MongoDB')
            return
        }

    const db = client.db('frutasDB')
    const frutas = await db.collection('frutas').find().toArray()
    await disconnectFromMongoDB()
        res.json(frutas)
}
```

las cuales serán utilizadas en el módulo de ruteo.

src/controllers/frutasController.js

```
module.exports = {
  getFrutas,
  getFrutaPorId,
  getFrutaPorNombre,
  getFrutaPorImporte,
  crearFruta,
  actualizarFruta,
  eliminarFruta,
}
```

Página actualizada 25 jun 2026 Denunciar abuso

denominada **frutasController**. Esta se transforma en un objeto contenedor de cada una de las funciones asincrónicas.

src/routes/frutasRoutes.js

```
const express = require('express')
const router = express.Router()
const frutasController = require('../controllers/frutasController')

// Rutas de frutas
router.get('/', frutasController.getFrutas)
router.get('/:id', frutasController.getFrutaPorId)
router.get('/nombre/:nombre', frutasController.getFrutaPorNombre)
router.get('/importe/:precio', frutasController.getFrutaPorImporte)
router.post('/', frutasController.crearFruta)
router.put('/:id', frutasController.actualizarFruta)
router.delete('/:id', frutasController.eliminarFruta)

module.exports = router
```



Arquitectura y Documentación …

MiddleWares y la ruta general de nuestro proyecto.

src/server.js

```
const express = require('express')
const app = express()
const PORT = process.env.PORT || 3000

const frutasRoutes = require('./src/routes/frutasRoutes')

app.use(express.json())
app.use((req, res, next) => {
  res.header("Content-Type", "application/json charset=utf-8")
  next()
})

app.use('/frutas', frutasRoutes)

app.listen(PORT, () => console.log(`API de frutas escuchando en ${PORT}`))
```

Express Router es una poderosa herramienta que nos permite organizar y modularizar nuestras rutas de manera eficiente. Al adoptar una estructura de carpetas lógica y separar las rutas y controladores, logramos una aplicación más limpia, mantenible y escalable, facilitando la separación de responsabilidades, lo que permite a los desarrolladores trabajar de manera más efectiva y colaborativa en equipos.

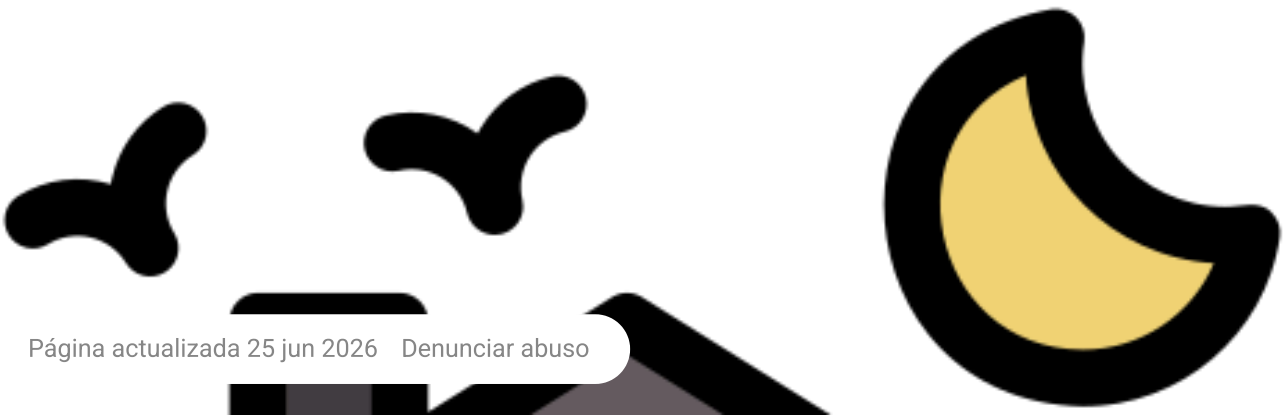
Además, el uso de middleware agrega capas adicionales de funcionalidad y validación a nuestras rutas. Su uso es esencial para construir aplicaciones web sólidas y bien organizadas.

La importancia de documentar

Página actualizada 25 jun 2026 Denunciar abuso

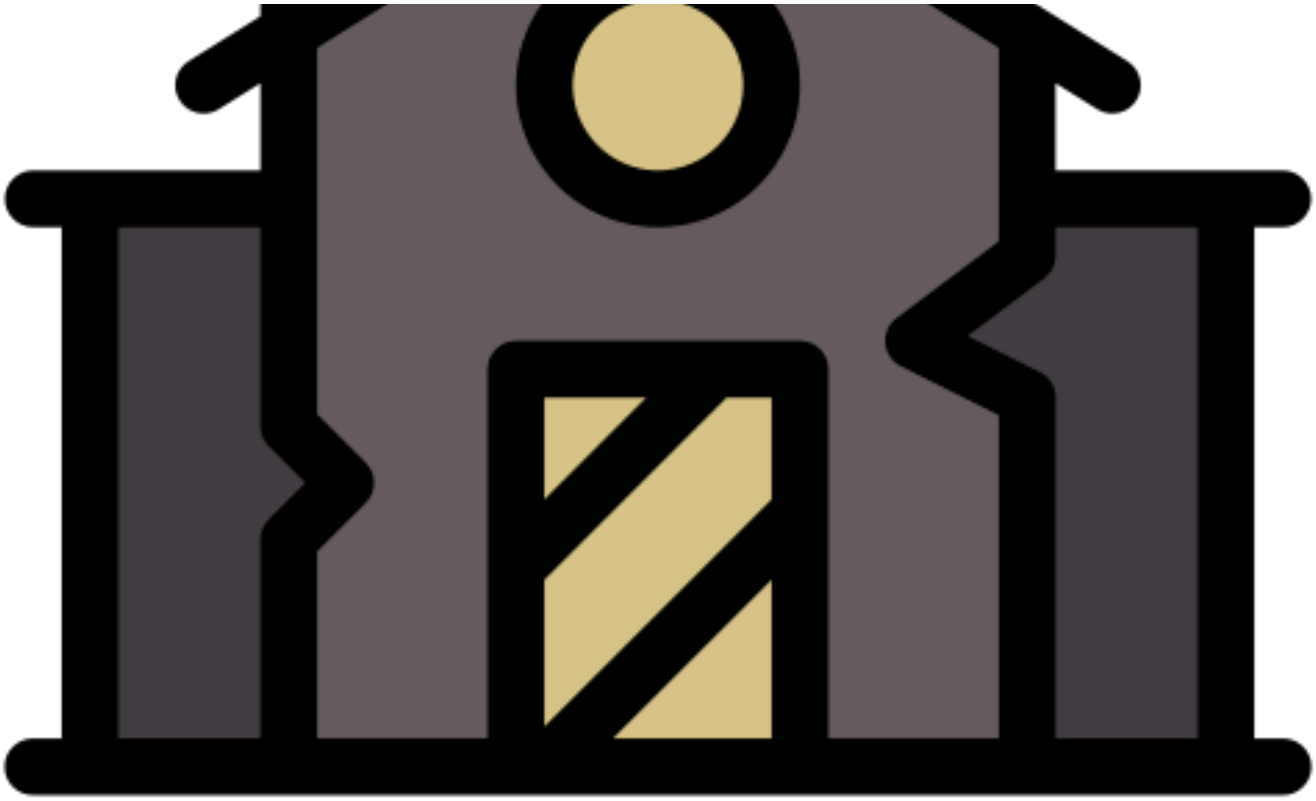
Arquitectura y Documentación …





Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …

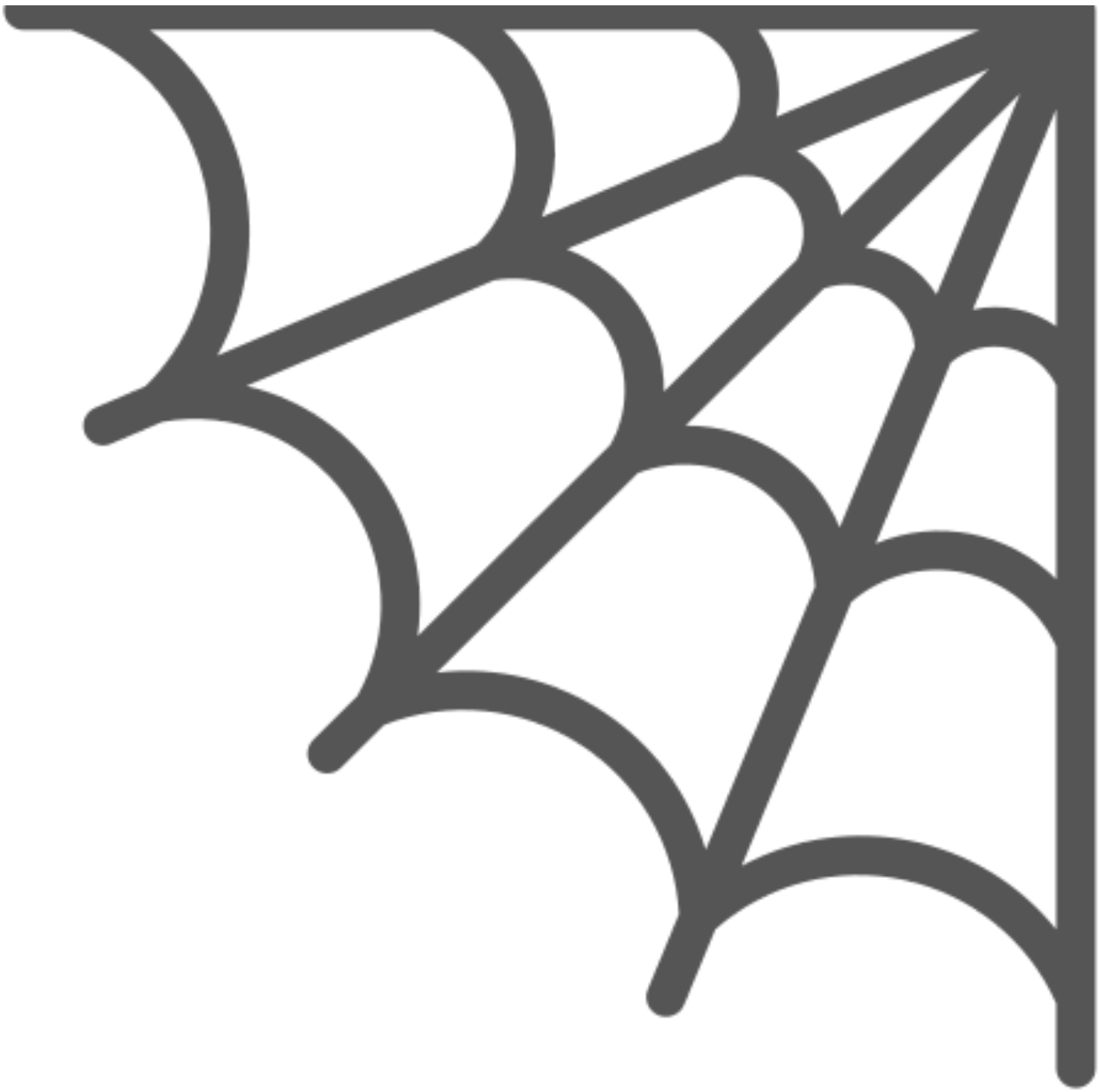


Llegamos al momento más terrorífico para cualquier desarrollador de software... ¡DOCUMENTAR!

😂

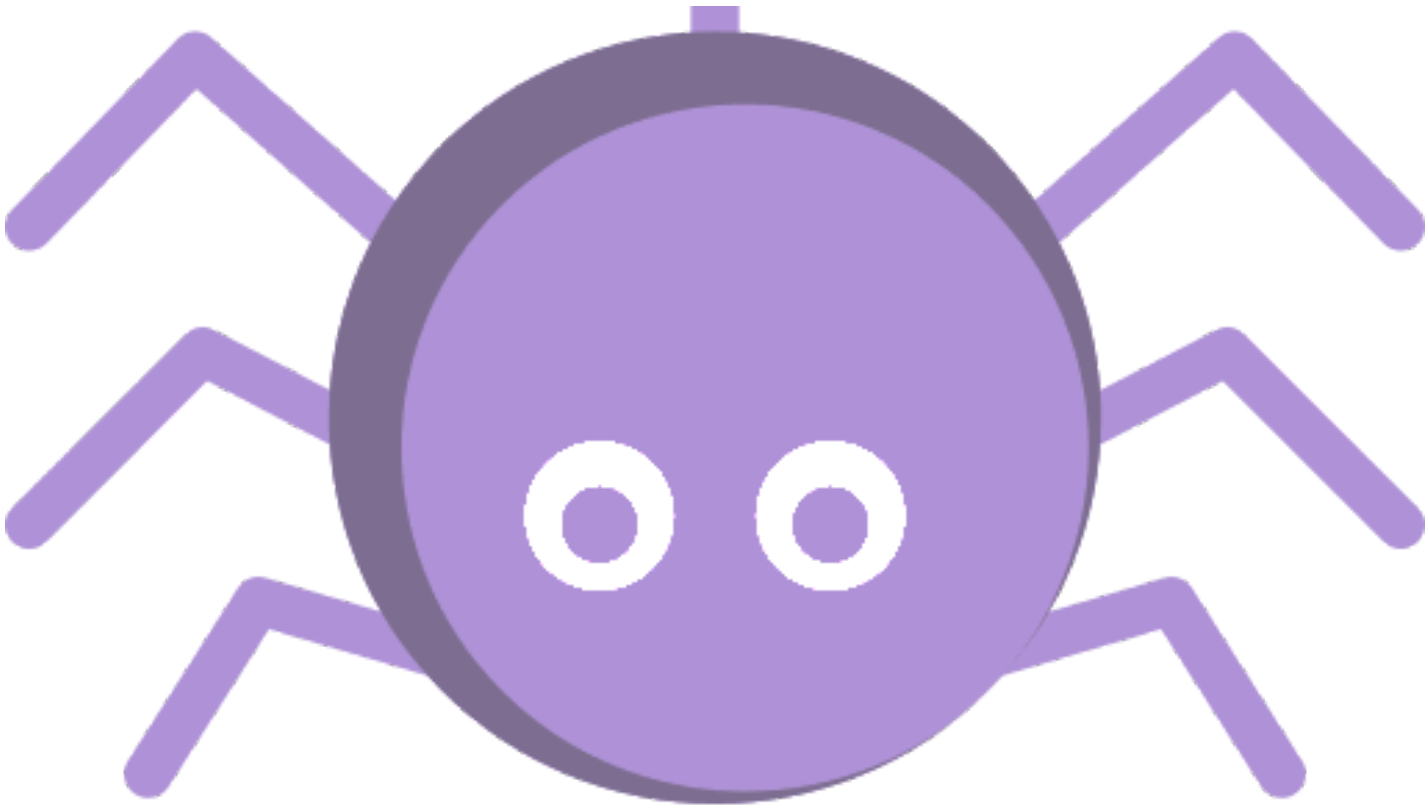
Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …



Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …



Página actualizada 25 jun 2026 Denunciar abuso

mismos cuentan con una guía, manual, o documento que explica su uso, el software no es la excepción a la regla.



Como parte de nuestra rutina cotidiana, debemos incluir la documentación de cada pieza de software que realicemos.

documentación en nuestros proyectos, son:

- Mejorar la comprensión del proyecto
- Facilitar la colaboración
- Reducir errores y retrasos
- Facilitar la transición y/o adopción
- Aportar en la planificación futura

Página actualizada 25 jun 2026 Denunciar abuso



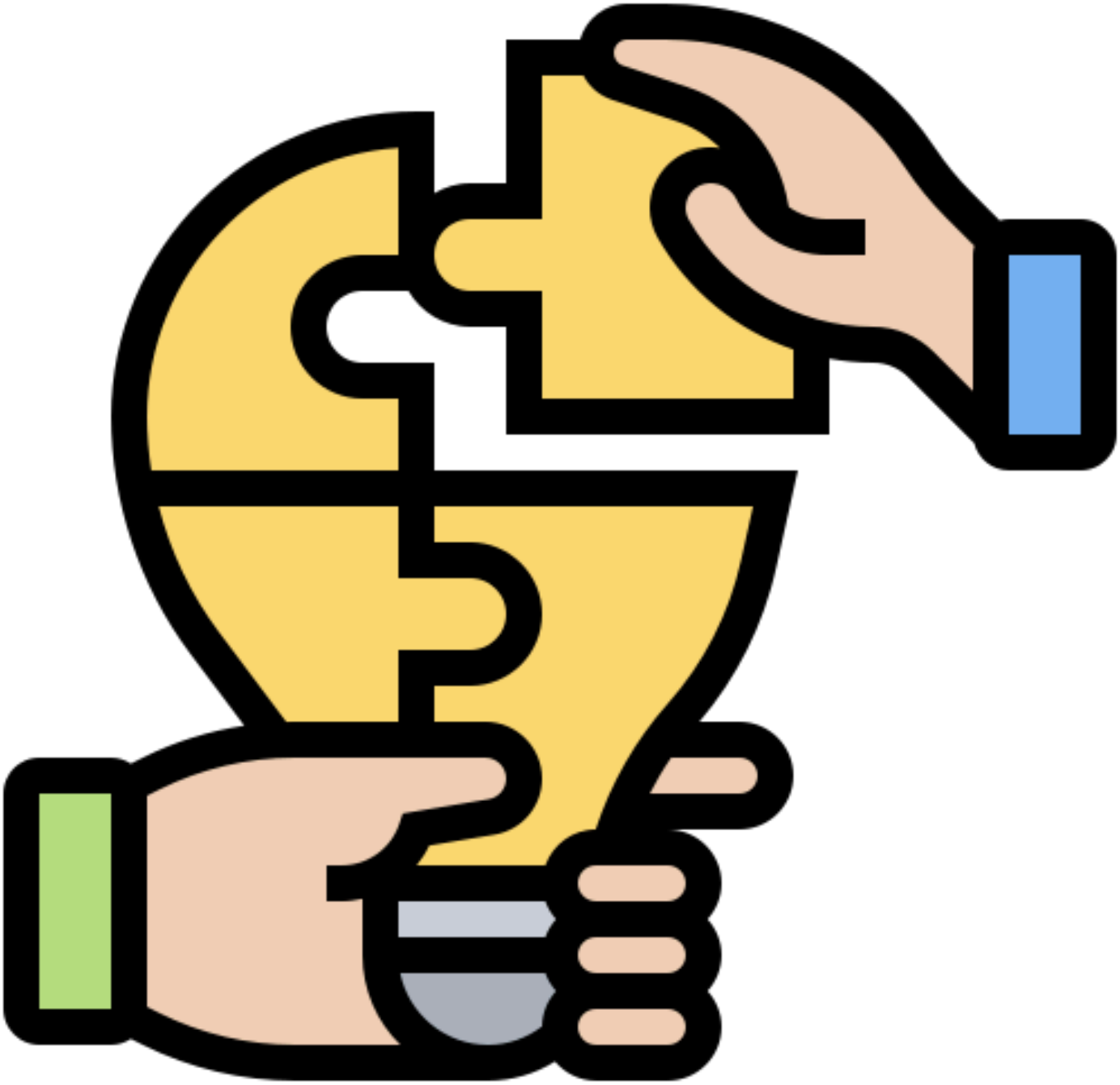
Mejorar la comprensión del proyecto

La documentación detallada del desarrollo de aplicaciones ayuda a los desarrolladores a comprender mejor el proyecto y sus objetivos, así como a identificar cualquier problema o desafío que puedan surgir.

Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …

Una buena documentación hace que sea más fácil para los miembros del equipo colaborar y trabajar juntos en el proyecto. Esto es especialmente importante cuando hay varios equipos de desarrollo trabajando en diferentes aspectos de la aplicación.



Página actualizada 25 jun 2026 Denunciar abuso



Reducir errores y retrasos

También ayuda a los equipos de desarrollo a evitar errores y retrasos innecesarios, ya que pueden referirse a la documentación para asegurarse de que están siguiendo pautas y procedimientos adecuados.

Arquitectura y Documentación …

Ante el constante cambio de recursos vinculados a un equipo de desarrollo o la contratación de nuevos integrantes, la documentación detallada es muy útil para acelerar cualquiera de estos tipos de transiciones.



Aportar en la planificación futura

La documentación detallada puede ser útil para futuros proyectos similares, ya que puede servir como base para la planificación y el desarrollo de otras futuras aplicaciones, o para incrementar las capacidades de la aplicación actual.

Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …

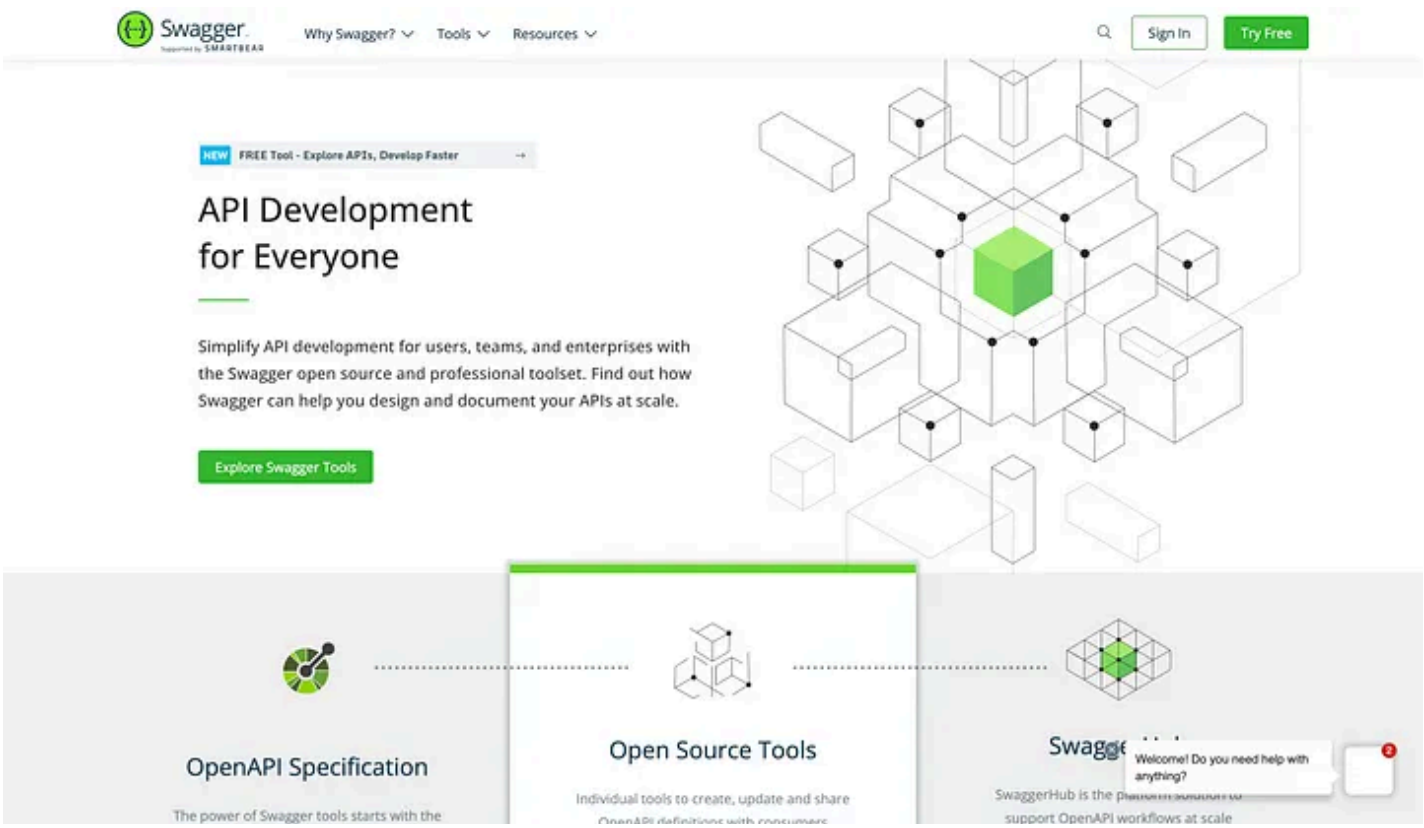
Hay varias herramientas que puedes utilizar para documentar aplicaciones Node.js.

Veamos a continuación algunas de las más populares que pueden servirnos para investigar con mucho más tiempo.

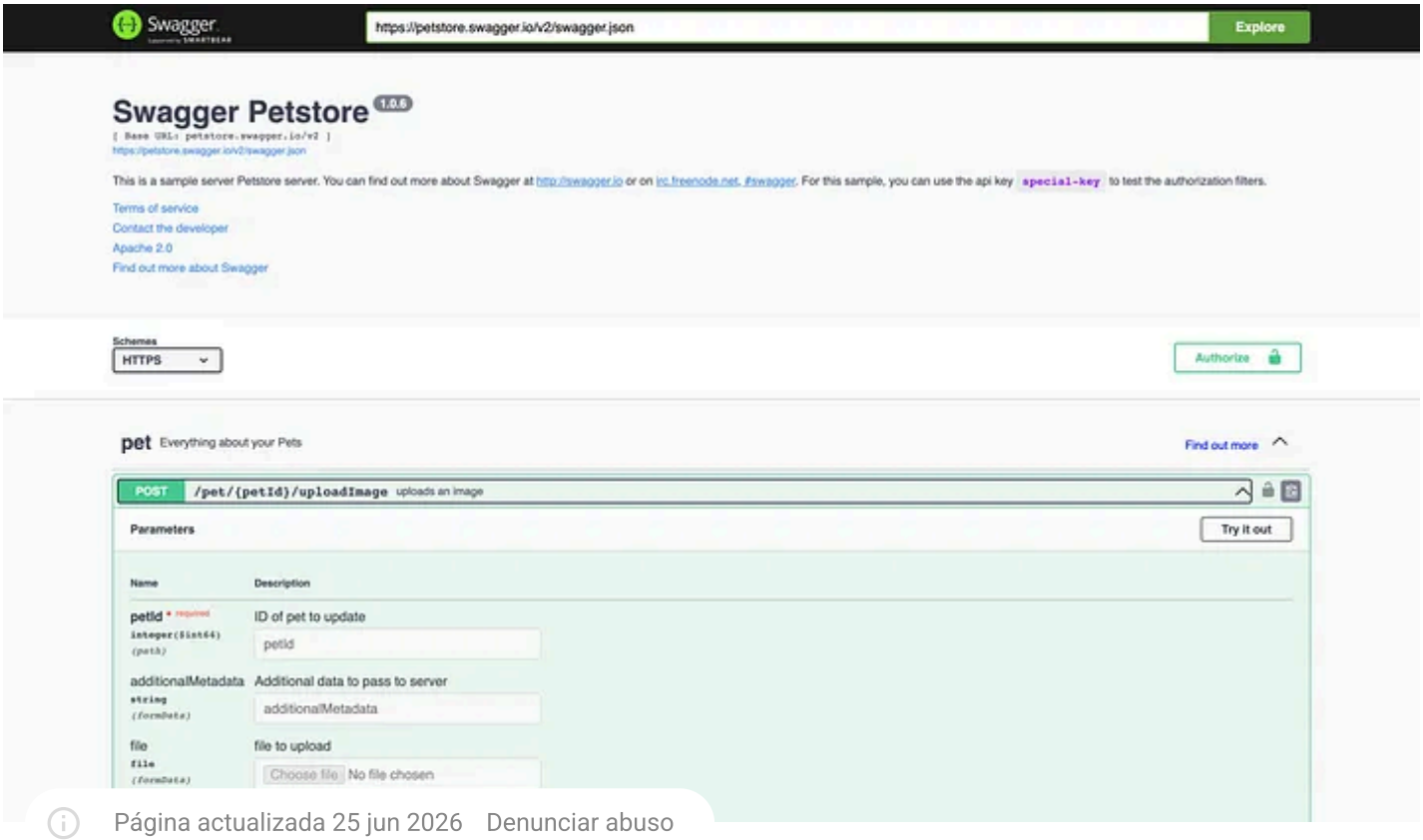


Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …



El primero en la lista para crear documentación API es SwaggerUI por Smartbear. Su uso es totalmente gratuito. Los consumidores de API pueden comprender fácilmente la API comprobando rápidamente qué puntos finales llevan a cabo qué funciones utilizando SwaggerUI. Sin desarrollar la lógica, se puede interactuar con la API utilizando SwaggerUI.



Página actualizada 25 jun 2026 Denunciar abuso

operaciones, parámetros y respuestas de la API, junto con ejemplos de código interactivo. Swagger UI se integra perfectamente con Swagger Hub y admite varios formatos de salida, como HTML, Markdown y PDF.

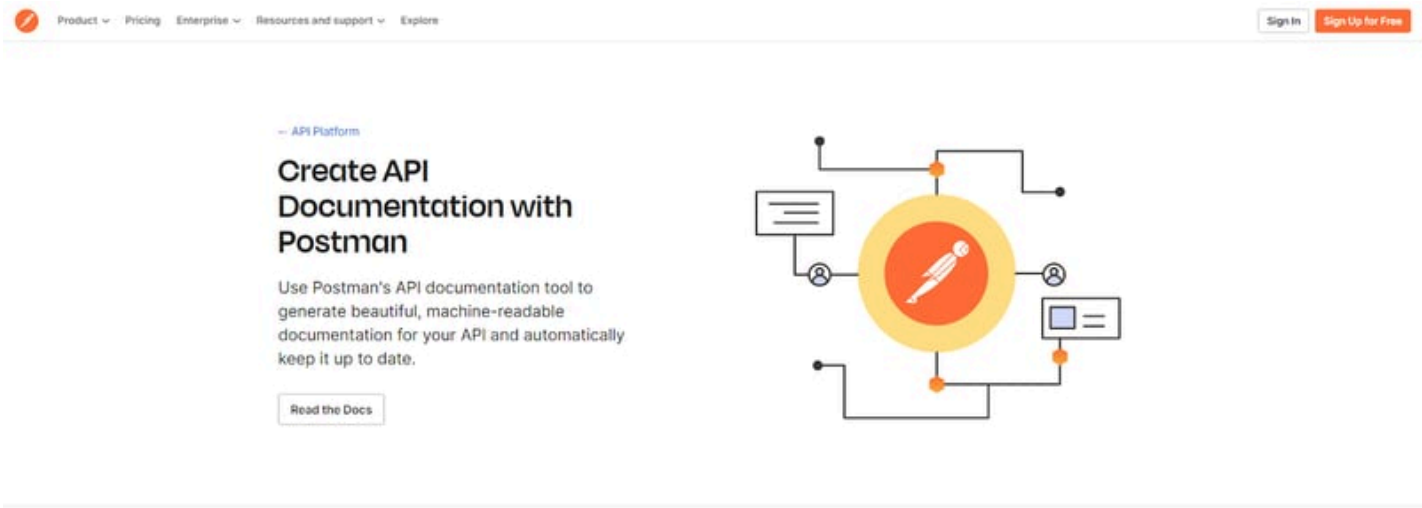
Características principales

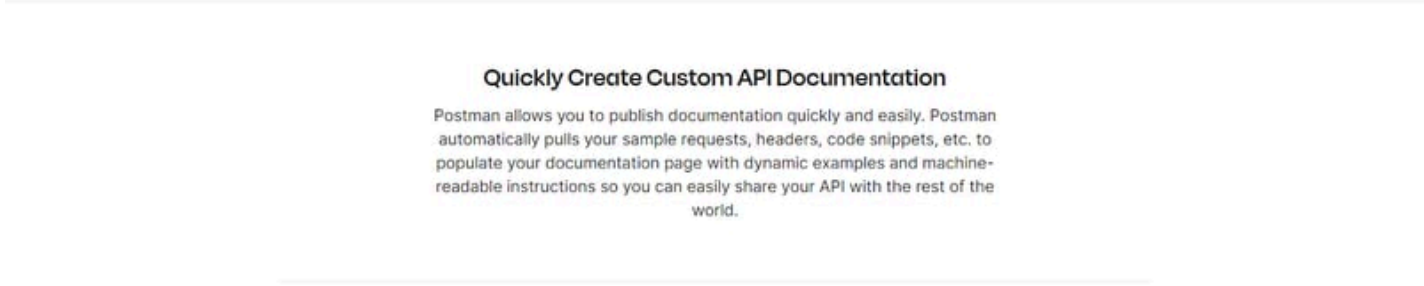
- Interfaz interactiva: Swagger UI proporciona una interfaz interactiva para la exploración y prueba de API. Los desarrolladores pueden explorar puntos finales, métodos, cargas útiles de solicitud/respuesta y otros detalles esenciales de una API directamente desde la interfaz de usuario.
- Documentación clara: Swagger UI genera documentación clara y fácil de leer a partir de su especificación OpenAPI. Presenta los recursos de la API en un formato fácil de usar, haciéndola accesible tanto para la implementación en el back-end como para el consumo en el lado del cliente.
- Soporte de autenticación integrado: Es compatible con varios métodos de autenticación de API. Los desarrolladores pueden probar puntos finales con diferentes mecanismos de autenticación, garantizando interacciones API seguras.
- Generación automática de SDK: Swagger UI genera automáticamente SDK de cliente de API para varios lenguajes de programación. Esta característica simplifica la integración para los desarrolladores que utilizan diferentes pilas de tecnología.

Precios: Swagger UI viene con la versión gratuita para un solo usuario y ofrecen $23/mes para un equipo.

Ejemplo:https://editor.swagger.io/#/

Postman





Página actualizada 25 jun 2026 Denunciar abuso

rápidamente, extrayendo automáticamente solicitudes de ejemplo, fragmentos de código y cabeceras de colecciones de API. El botón «Ejecutar en Postman» simplifica el uso compartido, y la función de comentarios facilita la colaboración dentro de los equipos. Mantenga informados a sus usuarios con actualizaciones en tiempo real y una comunicación organizada mediante Postman.

Características principales

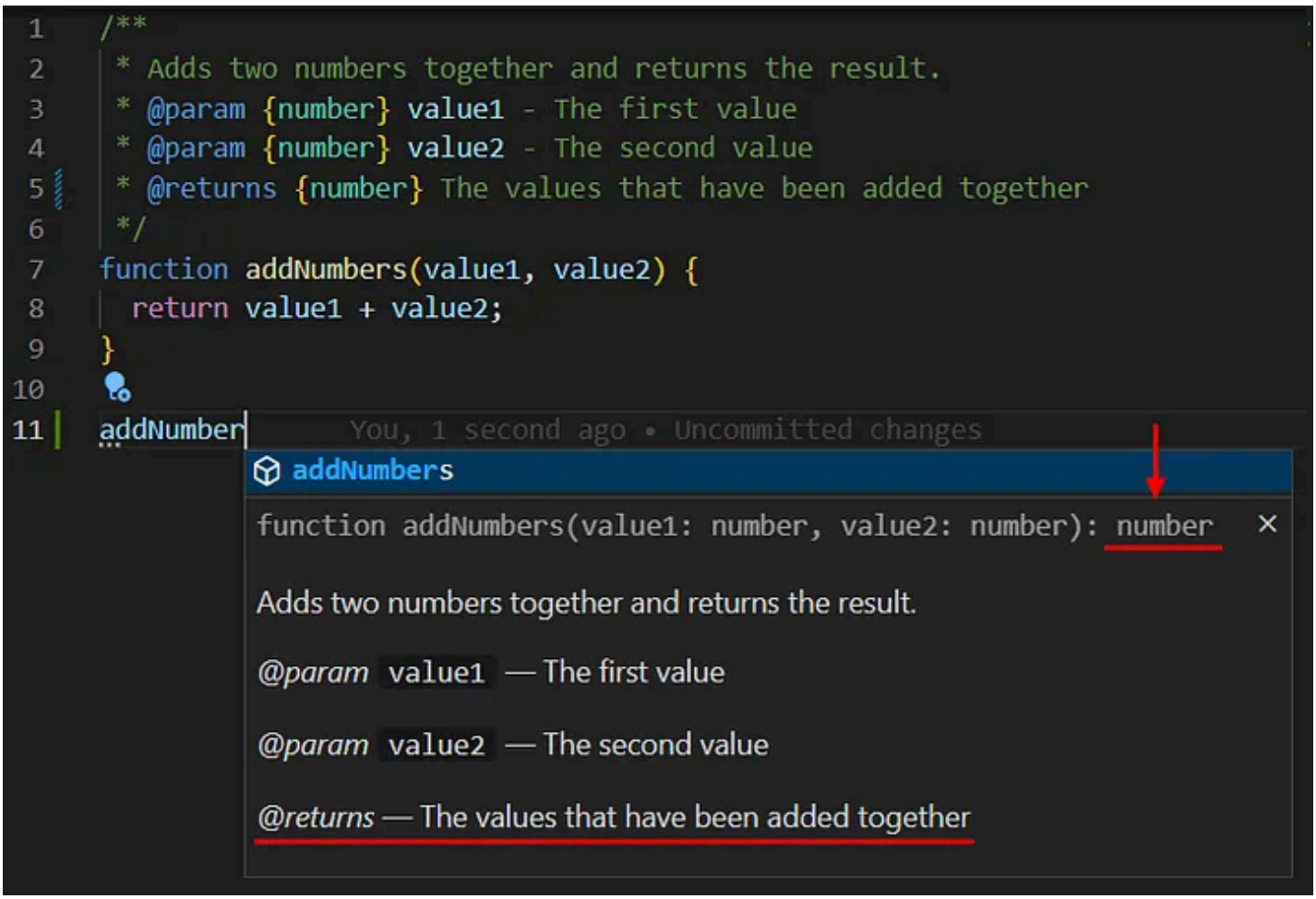
- Generación automática de documentación: Postman genera automáticamente documentación de API a partir de sus definiciones de OpenAPI 3.0. Incluye detalles sobre las solicitudes, código de muestra en varios lenguajes de cliente y ejemplos dinámicos.
- Actualizaciones en tiempo real: Nunca dejes que tu documentación vaya por detrás de tu servicio. Postman actualiza su documentación en tiempo real cada vez que realiza cambios en su colección de API.
- Colaboración y comentarios: La función de comentarios de Postman permite a los equipos colaborar eficazmente. Comparta comentarios, organice cambios y notifique errores o actualizaciones a sus compañeros de equipo.
- Legible y consumible: La documentación de Postman está diseñada para todo el mundo: desarrolladores, probadores, jefes de producto y socios. Su formato legible por humanos permite el consumo directo a través de las instancias de Postman.

Precio: Postman tiene una versión gratuita y sus planes de pago comienzan en 14 dólares al mes.

Ejemplo: https://documenter.getpostman.com/view/9625258/SzS8tQrQ

JS Docs

Arquitectura y Documentación …



JSDocs es también un generador de documentación API que escanea archivos y genera documentación basada en la documentación JSDocs añadida a los archivos.

Global

Home

Global

addNumbers
Greeting
useGreeting

Methods

addNumbers(value1, value2) → {string}

Adds two numbers together and returns the result.

Parameters:

Name	Type	Description
value1	number	The first value
value2	number	The second value

Source: utils/addNumbers.js, line 14

Returns:

The values that have been added together

Type
string

Example

```
const a = 10;
const b = 20;

const result = addNumbers(a, b);
console.log(result);
// Logs: 30
```

Greeting(props)

Markdown

***domines*).**

Será el que elegimos para crear documentos bien estructurados y con el formato apropiado, sin preocuparnos por la complejidad de lenguajes de marcado tradicionales como ser HTML, LaTeX u otras herramientas.

ligero que se utiliza comunmente para formatear el texto de una manera sencilla y facil de leer.



Fue creado por *John Gruber* en 2004 con el objetivo de que las personas escribieran documentos en formato fácil de leer y fácil de redactar, utilizando un formato de texto plano.

Markdown se ha vuelto muy popular entre los desarrolladores de software, los escritores técnicos, y otros usuarios que necesitan crear documentación de una manera rápida y sencilla.

Página actualizada 25 jun 2026 Denunciar abuso

como: encabezados, listas, enlaces, texto en negrita y cursiva, entre otros.

Soporta la inserción de imágenes, hipervínculos internos y externos, y cuenta con una capacidad avanzada para poder crear diagramas de diferentes tipos:

- de Clase
- UML
- de Entidad relacional

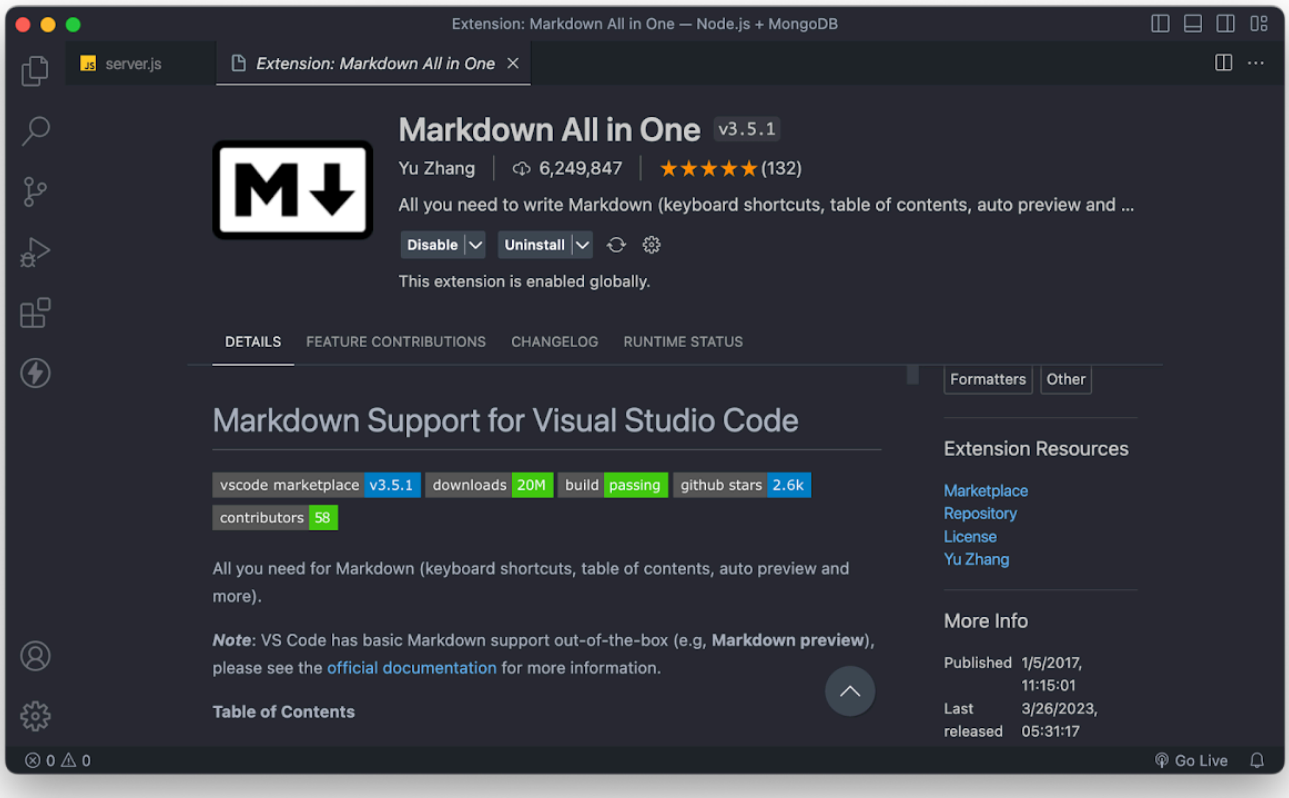
Entre otros. Para estos últimos, necesitaremos extensiones adicionales.

Arquitectura y Documentación …



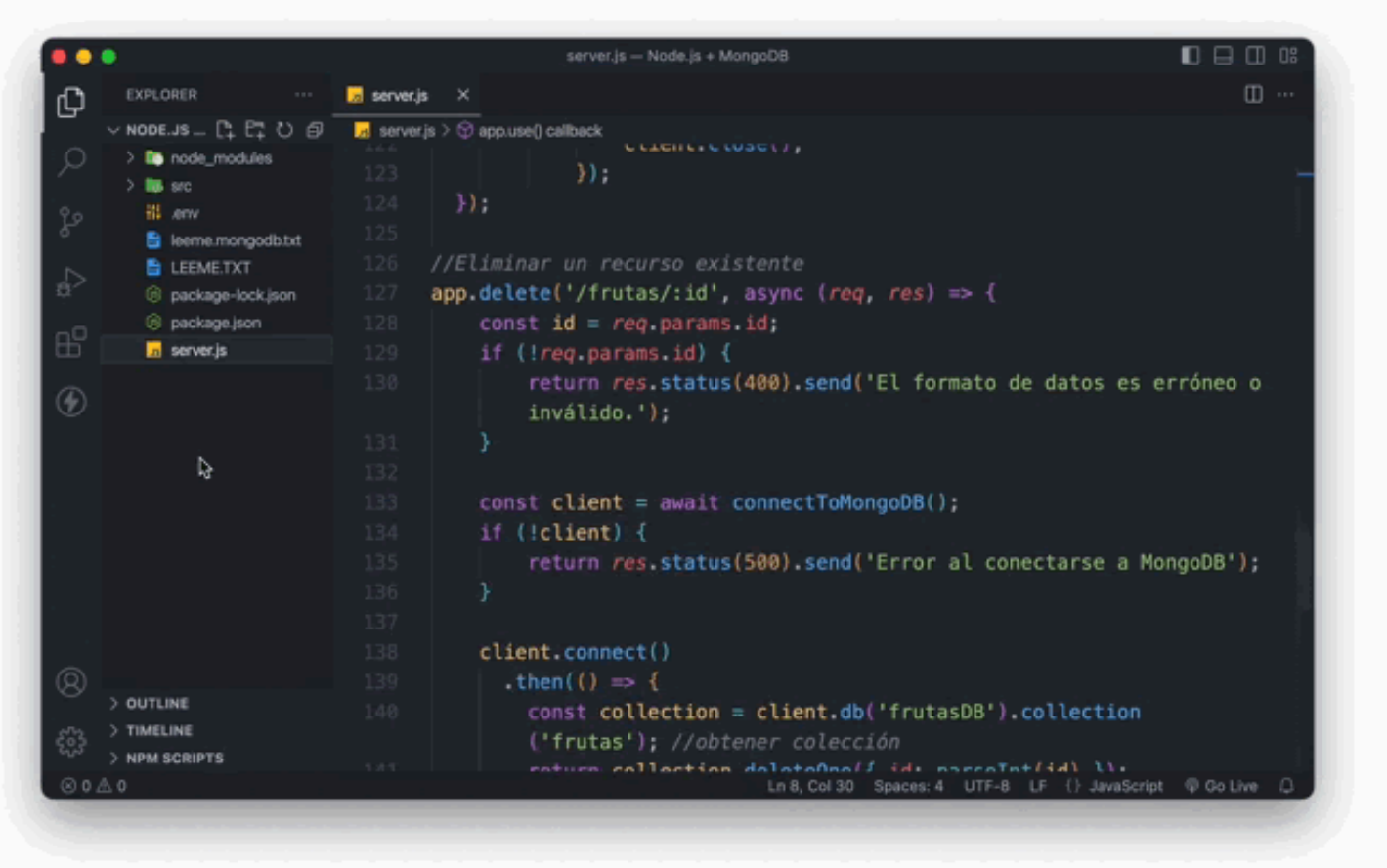
Página actualizada 25 jun 2026 Denunciar abuso

visualizador en tiempo real de archivos Markdown dentro de nuestro editor.



Arquitectura y Documentación …

- Markdown Preview Enhanced
- Markdown Preview GitHub Styling
- Markdown Preview



Prueba más adelante otras opciones.

Te ayudarán a conocer otras formas de generar documentación, y estar preparada para adoptar prácticas alternativas de equipos de trabajo.

Sintaxis de MarkDown

convertir en título. Debemos tener presente respetar el espacio entre este caracter y el título, para que se genere correctamente.

```Markdown
# Título de primer nivel
```

Cada hash **#** adicional indica un nivel de título adicional. Por lo tanto, el primer ejemplo crea un título de primer nivel y el último ejemplo crea un título de sexto nivel.

```Markdown
# Título de primer nivel
## Título de segundo nivel
### Título de tercer nivel
#### Título de cuarto nivel
##### Título de quinto nivel
###### Título de sexto nivel
```

formas diferentes:

1. utilizando caracteres underscore _
2. utilizando caracteres asterisco *

```
Este es un texto con __formato negrita__.
Este es otro texto con _formato cursiva_.
Este es un texto con ___formato combinado de negrita y cursiva___.

Este es un texto con **formato del tipo**.
Este es otro texto con *formato del tipo*.
Este es un texto con ***formato combinado de negrita y cursiva***.
```

Dos caracteres underscore o dos asteriscos = **negrita**

Un caracter underscore o un asterisco = *cursiva*

Tres caracteres underscore o tres asteriscos = ***negrita y cursiva***

```
1  # El lenguaje Markdown
2
3  Este es un texto con __formato negrita__.
4
5  Este es otro texto con _formato cursiva_.
6
7  Este es un texto con ___formato combinado de negrita y cursiva___.
8
9  Este es un texto con **formato del tipo**.
10
11 Este es otro texto con *formato del tipo*.
12
13 Este es un texto con ***formato combinado de negrita y cursiva***.
```

El lenguaje Markdown

Este es un texto con **formato negrita**.

Este es otro texto con *formato cursiva*.

Este es un texto con ***formato combinado de negrita y cursiva***.

Este es un texto con **formato del tipo**.

Este es otro texto con *formato del tipo*.

Este es un texto con ***formato combinado de negrita y cursiva***.

Y como Markdown no cuenta con un formato de subrayado, podemos utilizar como alternativa, el antiguo tag HTML **<u>**, para subrayar texto.

```Markdown
~~ Texto tachado por defecto ~~
<u> Texto subrayado por defecto </u>
```

El uso del símbolo **>** (*mayor que*) se utiliza para representar una cita dentro de un bloque de texto breve.

```
> Code, Eat, Sleep, Repeat

Code, Eat, Sleep, Repeat
```

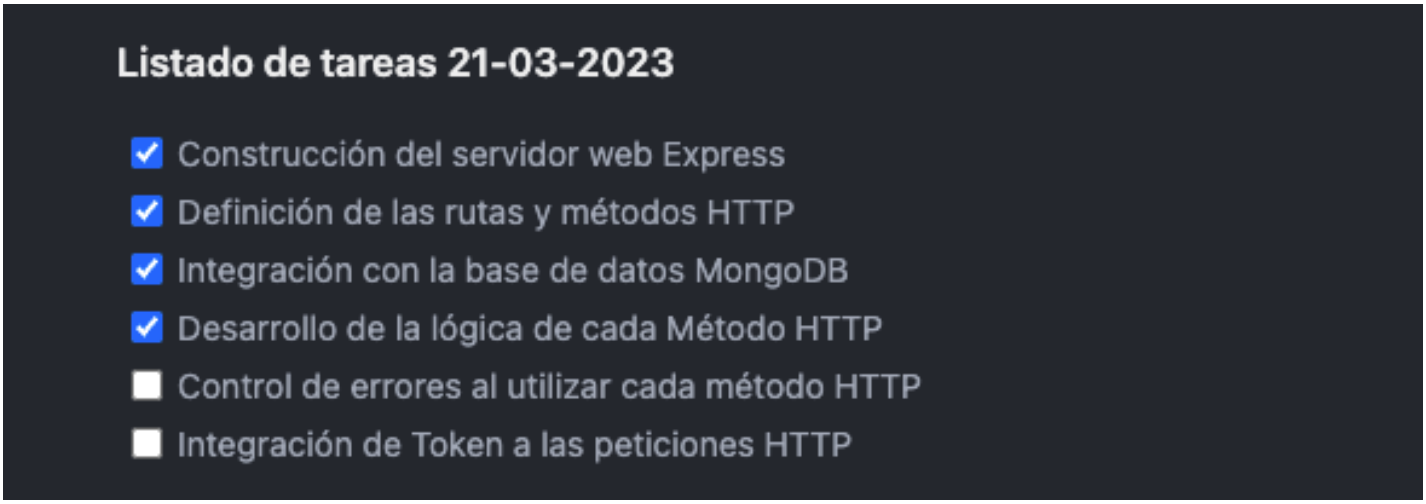
Listas de tareas y tablas

para contar con un resumen rápido dentro de la documentación, sobre qué funcionalidades están implementadas y cuáles no.

```Markdown
### Listado de tareas 21-03-2023

- [x] Construcción del servidor web Express
- [x] Definición de las rutas y métodos HTTP
- [x] Integración con la base de datos MongoDB
- [x] Desarrollo de la lógica de cada Método HTTP
- [ ] Control de errores al utilizar cada método HTTP
- [ ] Integración de Token a las peticiones HTTP
```

Estos checkboxes sólo se utilizan en modo de sólo lectura. No es posible tildarlos para cambiar su estado directamente desde el archivo Markdown como si fuese un formulario HTML.



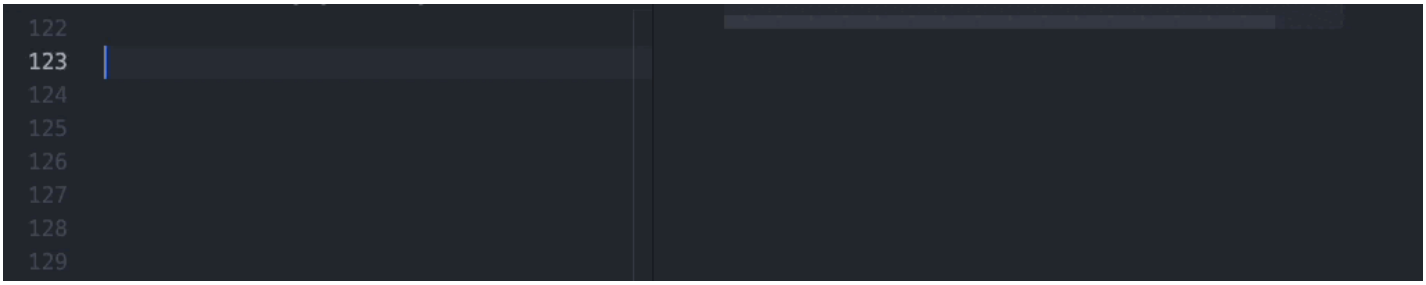
Página actualizada 25 jun 2026 Denunciar abuso

es combinando los guiones **-** para trazar las líneas horizontales y los caracteres Pipe **|** para establecer la estructura de columnas.

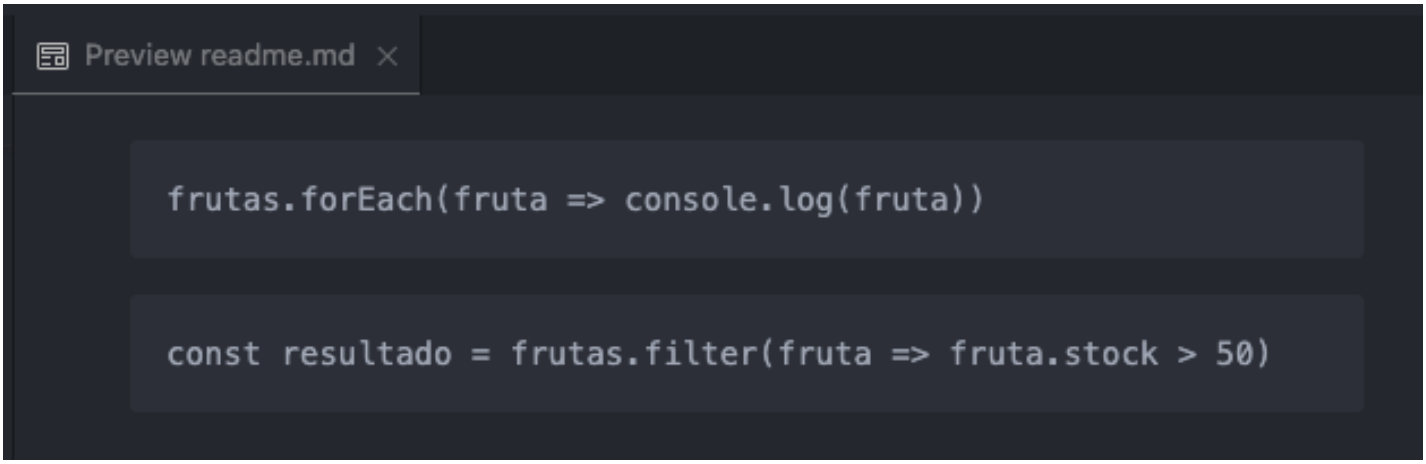
Como podemos ver en el código de ejemplo, podemos intercalar hipervínculos, como también cualquier otro juego de caracteres que formatee el texto a utilizar.

```
| PETICIÓN | URL | DESCRIPCIÓN |
|:--------:|-----|-------------|
|GET | [/frutas](/frutas)| Obtener todas las frutas |
| POST | [/frutas](/frutas) | Grabar una nueva fruta |
```

Además, contamos con la posibilidad de centrar el contenido de determinadas celdas, si así lo necesitáramos, integrando en los extremos de los guiones correspondientes a la celda a centrar, el caracter **:** (*dos puntos*).

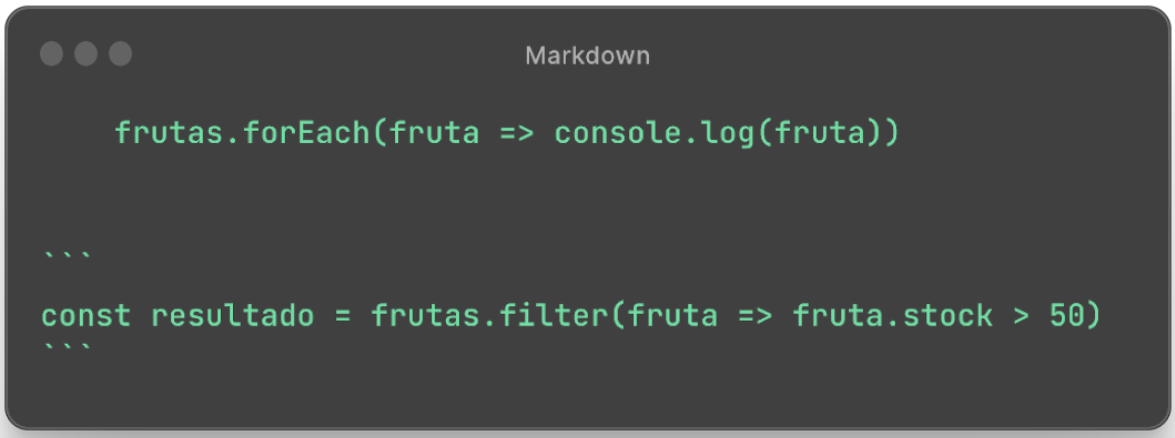


Agregar código



también contamos con dos formas de realizarlo:

1. dejando cuatro espacios delante de cada línea de código
2. Encerrando el o las líneas de código entre tres caracteres backtick



Agregar código con lenguaje



que el bloque de código coloree su sintaxis, tal cual en los editores de texto.



Referenciar hipervínculos

corchetes y paréntesis consecutivos, y anteponiendo el caracter **!**

Dentro de los corchetes definimos el texto alternativo para la imagen (*accesibilidad*), y dentro de los paréntesis, la ruta hacia la imagen en cuestión.

```Markdown
![Aprender Markdown](ruta-o-carpeta/nombre-de-imagen.jpg)

![Aprender Markdown](ruta-o-carpeta/nombre-de-imagen.jpg
'Texto o Tooltip sobre la imagen')
```

También, contiguo a la imagen definida, podemos sumar un comentario o texto del tipo **Tooltip** ó **Alt**, el cual aparecerá sobre la imagen al posicionar el mouse en esta. Para ello, debemos encerrarlo entre comillas simples.

```Markdown
![Pantalla con código backend](https://images.unsplash.com/photo-1627398242454.jpg
'Acceso a base de datos SQL con Node.js')
```

La inserción de hipervínculos externos también es posible realizar. Contamos con la opción de referencia directa del hipervínculo donde vemos la URL natural, y también la opción de referenciar un hipervínculo dentro de un texto o párrafo.

Página actualizada 25 jun 2026 Denunciar abuso

desordenadas, según nuestra necesidad. Dentro de los paréntesis indicamos el título o subtítulo del documento Markdown, reemplazando los espacios del título por guiones, usando minúsculas, y anteponiendo el caracter hash.

Markdown

```
- [El lenguaje Markdown](#el-lenguaje-markdown)
  - [Índice de contenidos](#índice-de-contenidos)
  - [Diagrama de flujo con Markdown y Mermaid:](#diagrama-de-flujo-con-markdown-y-mermaid)
  - [Diagrama de flujo condicional](#diagrama-de-flujo-condicional)
  - [Peticiones HTTP](#peticiones-http)
    - [Método POST](#método-post)
    - [Método PUT](#método-put)
    - [Método PATCH](#método-patch)
    - [Listado de tareas 21-03-2023](#listado-de-tareas-21-03-2023)
    - [Agregar una imagen en nuestro documento](#agregar-una-imagen-en-nuestro-documento)
```

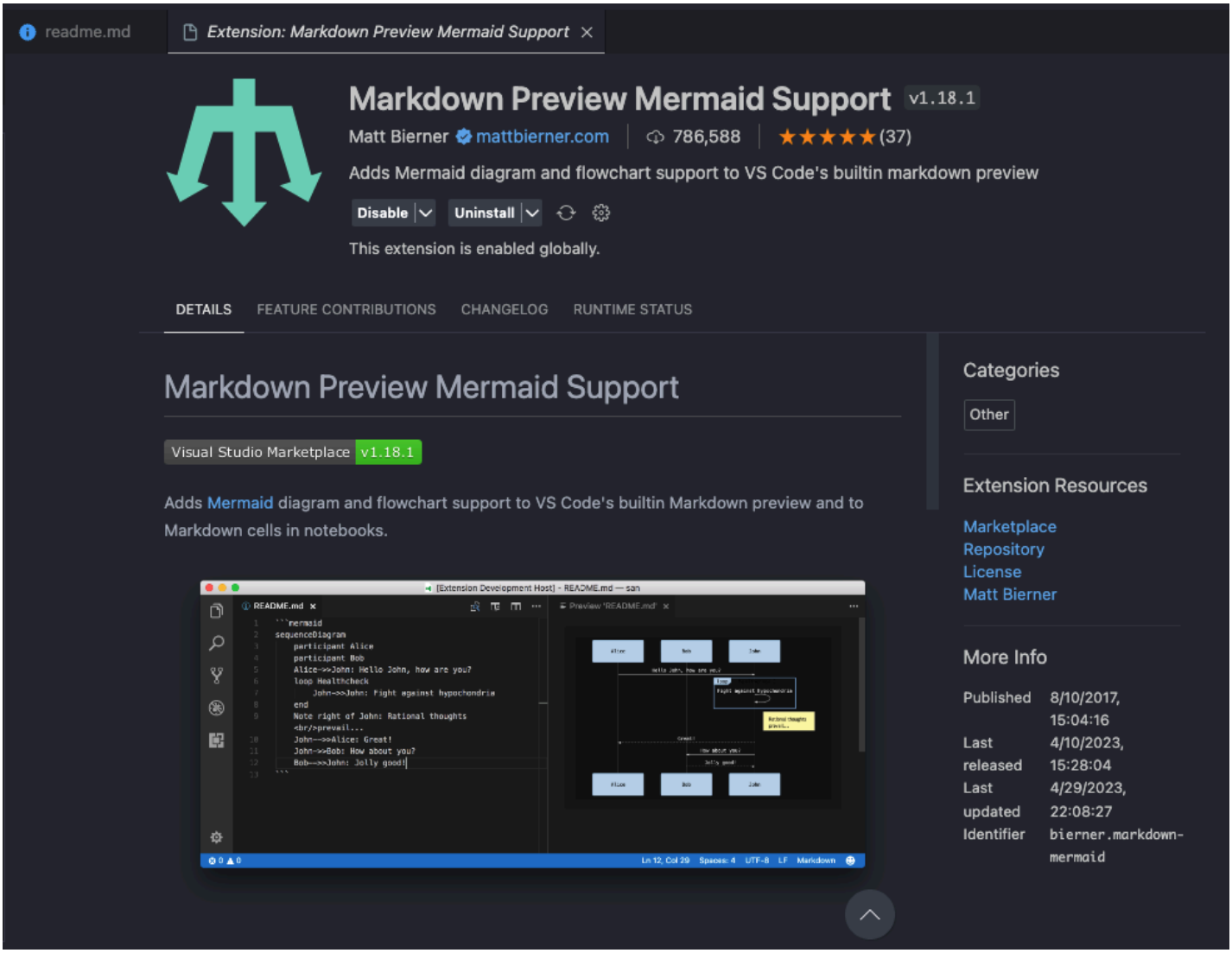
de su estructura interna, al nivel del título referenciado, generando la correspondiente tabulación de nivel.

Esto es una gran ayuda para no tener que luchar nosotras mismas con estos detalles.



agregar diagramas

En VS Code contamos con la extensión Markdown Preview Mermaid Support la cual nos facilita la integración de estos dentro del documento .md.



como si se tratase de código.

Además, debemos indicar como lenguaje de programación **‘mermaid’**, y definir a continuación el tipo de gráfico a construir.

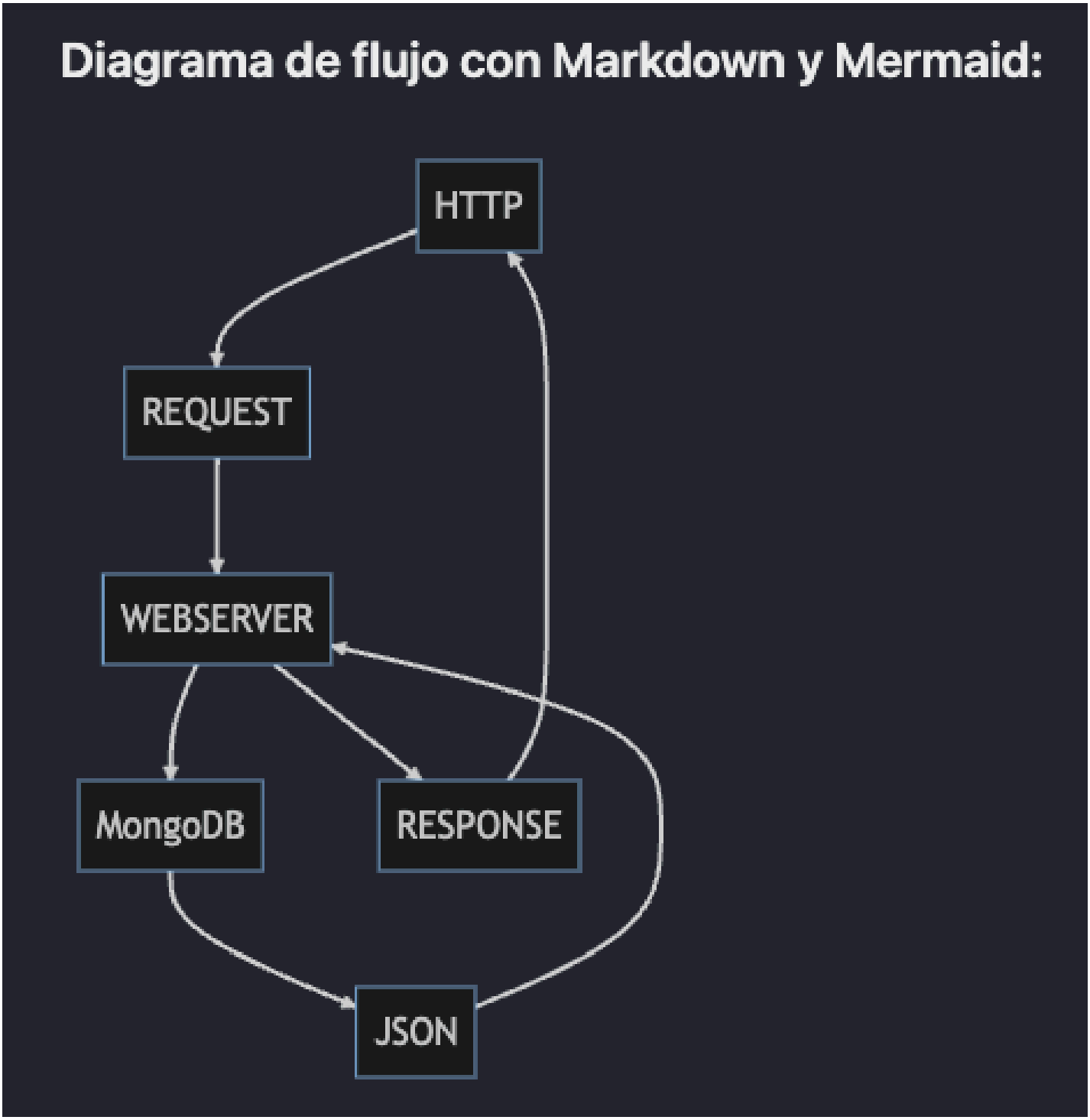
Markdown

````
## Diagrama de flujo con Markdown y Mermaid:

```mermaid
graph TD;
    HTTP-->REQUEST;
    REQUEST-->WEBSERVER;
    WEBSERVER-->MongoDB;
    MongoDB-->JSON;
    JSON--->WEBSERVER;
    WEBSERVER-->RESPONSE;
    RESPONSE-->HTTP;
```
````

de a poco diferentes tipos de diagramas.

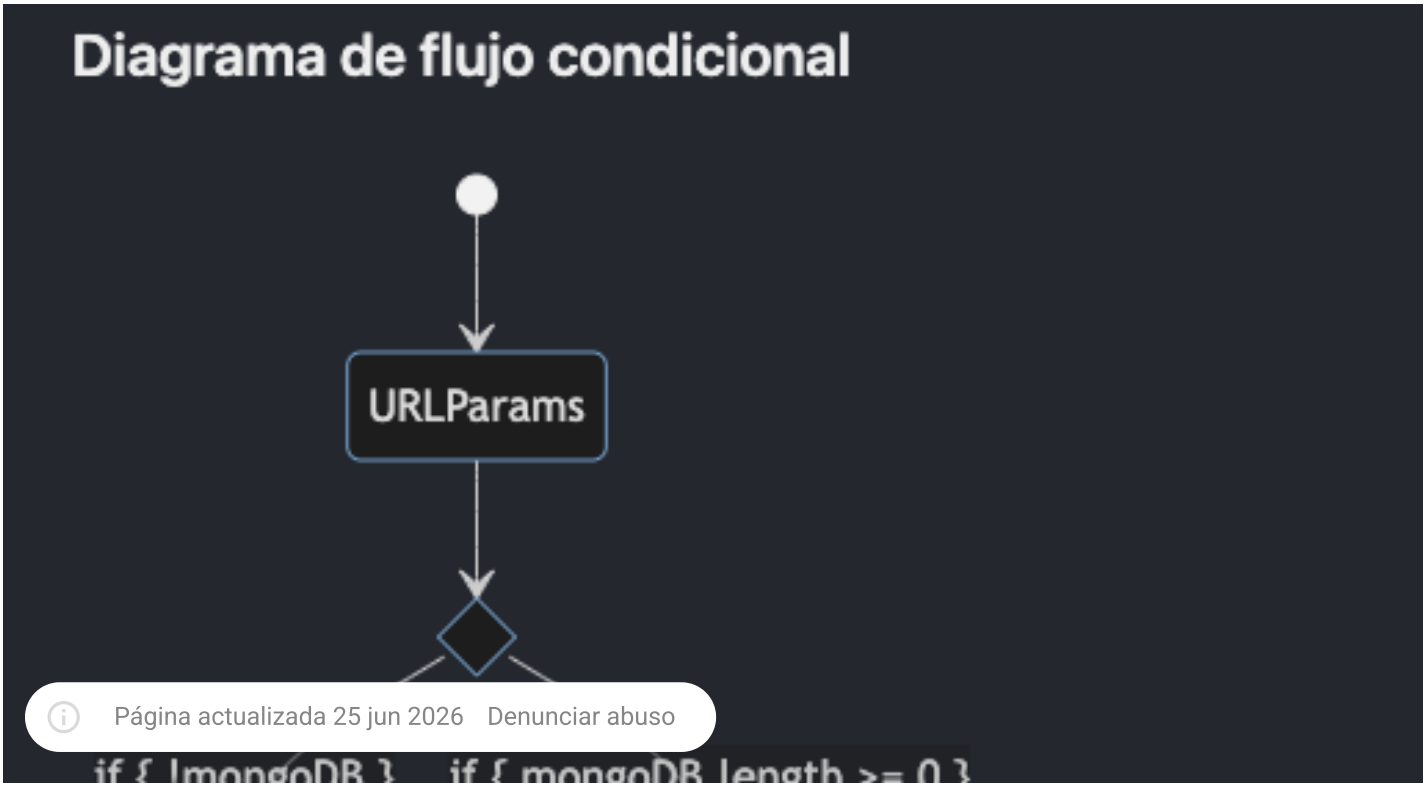
Aquí un ejemplo del resultado del código anterior.

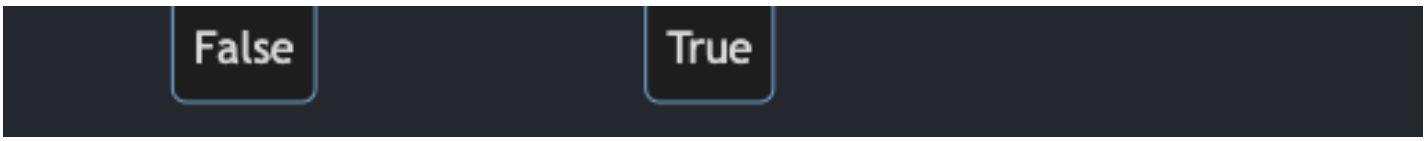


Diagramas de Flujo, *Entidad-Relación*, *de Gantt*, *de Secuencia*, *de Redes*, *de Clases*, entre otros.

Markdown

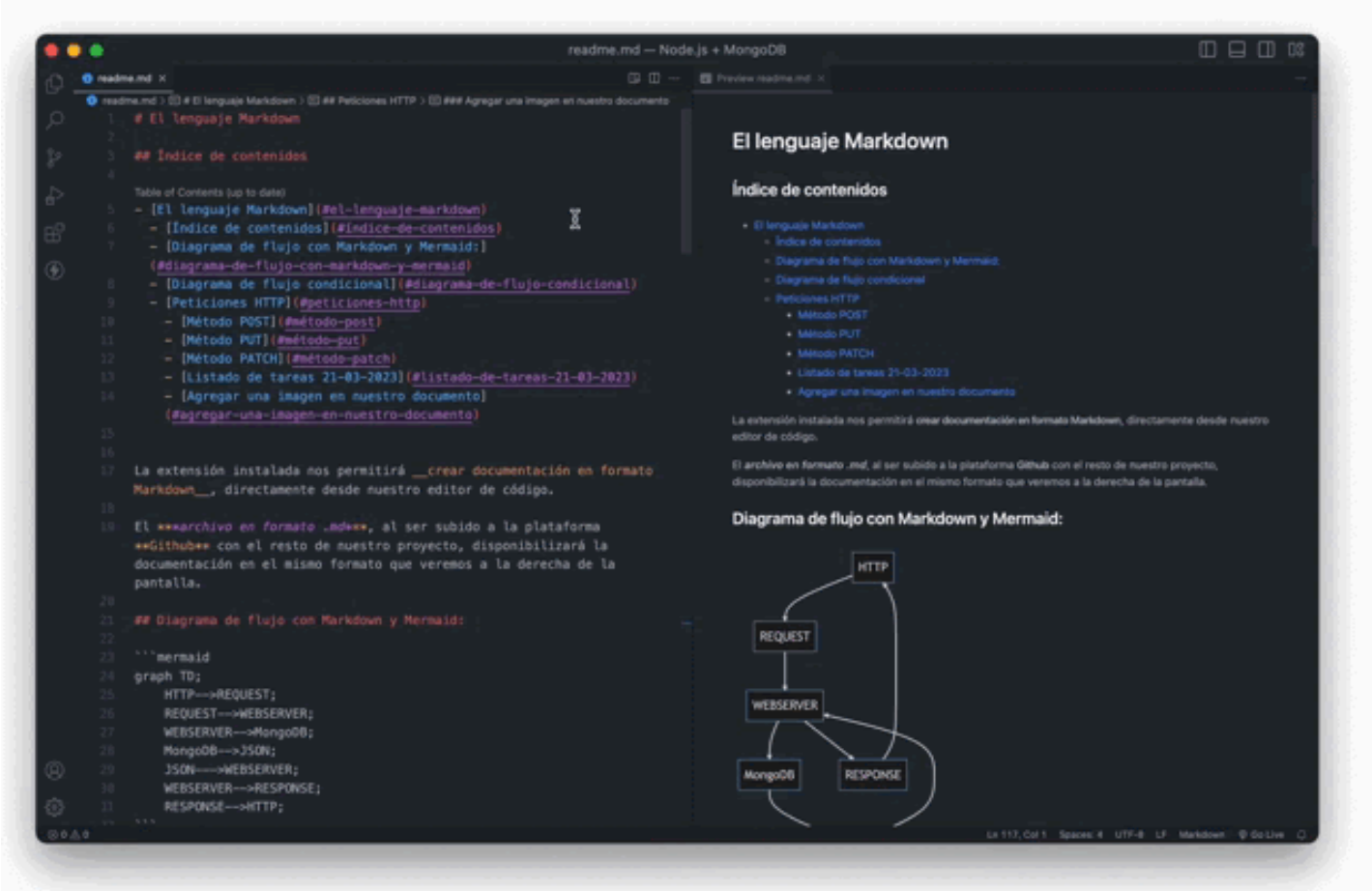
```
## Diagrama de flujo condicional
```mermaid
stateDiagram-v2
    state if_state <<choice>>
    [*] --> URLParams
    URLParams --> if_state
    if_state --> False: if { !mongoDB }
    if_state --> True : if { mongoDB.length >= 0
}
```
```





Como podemos apreciar en este repaso, Markdown es un lenguaje de marcado ideal para construir documentación efectiva y completa.

Va mucho más allá de poder estilizar un texto, y referenciar algún hipervínculo y/o imagen simple.



Estructura de la documentación API RESTFUL

través de un modelo que describa:

- El objetivo de la documentación
- Cuántos endpoints posee la API
- Detallar el rol de cada endpoint
- Proporcionar ejemplos de uso
- Detallar información adicional
- Actualizar la documentación

El objetivo de la documentación

Antes de crear la documentación, debemos tener claro el objetivo de la misma:

- ¿Qué información necesita el equipo frontend para utilizar los endpoints de nuestra aplicación backend?
- ¿Qué nivel de detalle debemos incluir en la documentación?
- ¿Qué tipo de herramienta utilizaremos para crearla?

Cuántos endpoints posee la API

El siguiente paso es que podamos identificar todos los endpoints de la API, y sus correspondientes métodos HTTP: (GET, POST, PUT, DELETE, etc.). Qué parámetros se requieren y qué respuesta obtendrán con cada uno de ellos.

Básicamente, detallar el camino feliz de la funcionalidad de cada endpoint.

Detallar el rol de cada endpoint

Una vez especificados los endpoints de la API, es importante documentar cada uno de ellos.

Para cada endpoint, es importante que incluyamos una descripción detallada de qué hacen, los parámetros necesarios para que respondan correctamente, el formato en el cual nos retornan cada respuesta, como también los posibles errores y condiciones bajo las cuales se devuelven estos errores.

Proporcionar ejemplos de uso

Página actualizada 25 jun 2026 Denunciar abuso

Es recomendable incluir ejemplos de uso para cada endpoint. El objetivo de dichos

parámetros.

También es clave detallar dentro de la documentación cómo interpretar la respuesta del endpoint.

Detallar información adicional

Puedes incluir información adicional en la documentación, como la versión de la API, la autenticación requerida para utilizar la API, los límites de velocidad de la API, las políticas de privacidad y seguridad, etc.

También es importante aclarar cualquier limitación de URLs para acceder a la API, entre otros puntos claves relacionados a la seguridad; algo de suma importancia hoy en día.

Actualizar la documentación

Es importante mantener la documentación actualizada, ya que cualquier cambio en la API debe ser reflejado en la documentación para evitar confusiones o errores.

Lo ideal es evolucionar la documentación, a la par de cada etapa de construcción de los endpoints. Esto hace más fácil su elaboración, y menos pesado nuestro día a día dado que diversificamos nuestras tareas cotidianas.

Página actualizada 25 jun 2026 Denunciar abuso

Arquitectura y Documentación …

Página actualizada 25 jun 2026 Denunciar abuso